

Intro to ML

24 March 2026 17:35

I forgot the order of the lectures so let's start with this.

AI supersedes machine learning. ML is one way of doing AI.

Artificial Intelligence, or AI, means building systems that appear to perform tasks requiring intelligence.

ML is a subfield where the system improves from data or experience rather than being fully programmed by hand.

AI = the goal: make machines behave intelligently.

ML = a method: let machines learn patterns from data.

The Turing test is a proposed test for whether a machine can behave intelligently in conversation.

A human judge communicates with two hidden participants: one is a human, and one is a machine. The judge can only interact with them through text or some indirect channel. If the judge cannot reliably tell which one is the machine, then the machine is said to have passed the test.

Supervised learning:

goal is to predict Y given X .

Eg predict which mail is spam or not, given a database of emails.

Unsupervised learning:

Goal is to understand the distribution of X , eg recognise which images are similar without being given labels, or to detect an outlier.

So in supervised learning the data includes both inputs and correct outputs and to learn the rule. Learn from examples with answers.

In unsupervised learning, the data only has inputs X with no given labels Y . Find patterns without answers.

Reinforcement learning:

An agent learns by taking actions and receiving rewards or penalties. Learning through trial and error.

Basics of Supervised Learning

09 June 2026 00:48

Predict Y given X .

When Y is discrete this is classification, when continuous this is regression.

For regression:

- given: joint distribution (X, Y) , $P_{X,Y}$
- $X \in \mathbb{R}^d$ are the features/predictors ...
- $Y \in \mathbb{R}$ is the label, outcome ...

$L(Y, f(X))$ measures the loss with some function f to predict Y from X .

goal : find f that minimizes risk $E[l(X, f(Y))]$

Risk is expected loss, or average loss over the population/distribution.

For classification:

- given: joint distribution (X, Y) , $P_{X,Y}$.
- $X \in \mathbb{R}^d$ are the features/predictors ...
- there are K classes, $Y \in \{1, \dots, K\}$

The K classes part just means that the label must be one of K categories.

goal: come up with some function g to predict the class Y from the features X . So min risk: $E[l(Y, g(X))]$

The simplest form of loss is if you are correct then 0, if you are wrong then 1.

Misclassification rate is the probability the classifier gets it wrong:

$E[1_{Y \neq g(X)}] = P[Y \neq g(X)]$ misclassification rate/probability of error

Three steps of statistical inference:

Identification - can you solve your problem with infinite data and computational power?

Estimation - how can you best approximate your solution with finite data and computational power?

Inference - can you quantify the statistical uncertainty of your solution.

For **regression**, consider the regression problem

$$\min_{f: \mathbb{R}^d \rightarrow \mathbb{R}} E[(Y - f(X))^2]$$

Solution is $f^*(x) = E[Y | X = x]$.

Meaning, if you are judged by squared error, then the best single number to predict is the average value of Y for people with the same features.

For instance, to predict how much someone spends on food with a £100k salary, your best guess is the average of everyone in the sample with the same features (£100k salary also).

For **classification**, consider the (Bayes) problem:

$$\min_{g: \mathbb{R}^d \rightarrow \mathbb{R}} \mathbb{P}[Y \neq g(X)]$$

The solution is the Bayes classifier:

$$g^*(x) = \arg \max_{k \in \{1, \dots, K\}} P[Y = k | X = x]$$

The risk of g^* is called Bayes error/risk.

The above is similar to regression, here we are saying that the best prediction is the most likely class given $X=x$. Look at the probability of each possible class then choose the class with the highest probability. Arg max means choose the option that gives the largest value.

In reality, we do not know the distribution of the data, instead we have a sample $1, \dots, n$. Instead of minimising the true expected error, which we cannot directly observe, minimise the error on the data we have.

For regression with squared error the true/test/generalisation error is: $E[(Y - f(X))^2]$

This is the error we care for, how the model performs on unseen data, but we have to use the **training error**:

$$\frac{1}{n} \sum_{i=1}^n (Y_i - f(X_i))^2$$

Average square error on the training data.

Test/generalization error -> Error on new unseen data

Training error -> Error on the data used to fit the model

KNN:

data $(X_i, Y_i)_{i=1, \dots, n}$ for a finite $n \in \mathbb{N}$, new point X^{test} with unknown label and you find the k nearest datapoints in the training data closest to it. Then use nearby points to make a prediction.

Note that wlog means without loss of generality:

Example: maybe the nearest points in the original dataset were points 7, 20, and 45.

Instead of writing:

$$(X_7, Y_7), (X_{20}, Y_{20}), (X_{45}, Y_{45})$$

we just rename them as:

$$(X_1, Y_1), (X_2, Y_2), (X_3, Y_3)$$

That is "without loss of generality."

For regression where Y is continuous, you predict the average of the nearby labels:

$$\hat{y} = \frac{1}{k} \sum_{i=1}^k Y_i$$

For classification, take the majority vote:

$$\hat{y} = \arg \max_{y \in \{1, \dots, K\}} \sum_{i=1}^k \mathbf{1}_{Y_i=y}$$

First, note that training error (also called empirical risk) is the average error a model makes on the exact dataset it was trained on.

A high training error indicates the model is too simple to capture the underlying patterns in the data (known as underfitting). Conversely, as model complexity increases, training error decreases, because the model learns to fit the training data more closely.

K in KNN controls the model's complexity.

If K=1 the model only looks at the single closest point. So the prediction can change a lot depending on tiny movements in X, the features of the point we are trying to classify or predict. The model is very flexible. If you get a training point as a test, then compare it to the point closes to it, which must be itself, then the training error is zero.

For classification, k=1 means every training point predicts its own label correctly, because its nearest neighbour is itself. Hence, **training** error is 0.

For K=n, the model looks at all training points, now no longer depends on X anymore, the input features.

For classification, it would just predict via the most common class in the whole training set.

This is not flexible enough. The idea is that:

Small k = very flexible model, low training error, risk of overfitting.

Large k = very simple model, higher training error, risk of underfitting.

- $k = n$: prediction does not depend on x

- training error: $\min_{y \in \{1,2\}} \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{Y_i=y}$

This is a binary classification between states 1 or 2.

Try always predicting class 1. Count how many mistakes you make.

Try always predicting class 2. Count how many mistakes you make.

Choose the class that gives fewer mistakes.

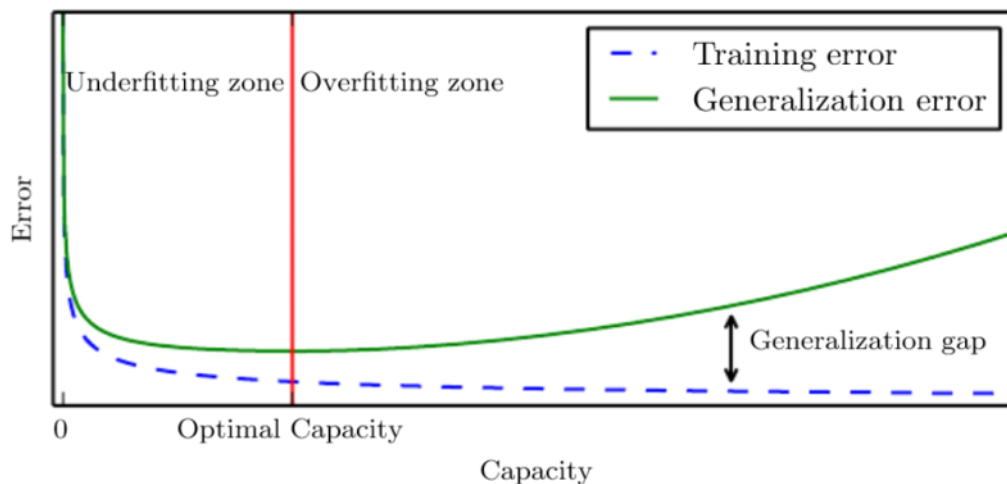


Figure 5.3 from the “Deep Learning” Book

The horizontal axis is model capacity, meaning how flexible or complex the model is. For k-nearest neighbours, capacity is controlled by k: small k means high flexibility; large k means low flexibility.

Higher capacity typically means higher complexity.

Capacity refers to a model's ability to fit a wide variety of functions and intricate data patterns.

Training error is a decreasing function because as the model becomes more flexible, it can fit the training data better.

Generalization error (out-of-sample error, expected loss/risk) is the error on new unseen data, what we actually care about.

In the underfitting zone, the model is too simple and cannot capture the real pattern. Eg $K=n$

Optimal capacity is where the model is flexible enough to capture the real pattern and not noise.

Overfitting zone means the model is too flexible, captures noise. Eg $K=1$

The generalization gap is the difference between training error and generalization error.

The next slide goes into why training error is usually too optimistic, meaning training error often makes the model look better than it really is on new data.

Let $X_1, \dots, X_n \in \mathbb{R}^d$ be a set of features and $P_{Y|X}$ be a conditional distribution of labels given features. For each X_i , draw a “training” label Y_i and an independent “generalization” label $\tilde{Y}_i$ from $P_{Y|X_i}$. Denote the training data by $\mathcal{T}$ and the generalization data by $\mathcal{G}$. Using the training data, a prediction function $\hat{f}(\cdot)$ has been produced for quadratic loss.

Then the expected optimism of the training error is

$$\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{\mathcal{T}} \left[\mathbb{E}_{\mathcal{G}} \left[\left(\tilde{Y}_i - \hat{f}(X_i) \right)^2 \middle| \mathcal{T} \right] - \left(Y_i - \hat{f}(X_i) \right)^2 \right] = \frac{2}{n} \sum_{i=1}^n \text{Cov}(Y_i, \hat{f}(X_i)).$$

For the same features $X_1, \dots, X_N$, two kinds of labels.

Y_i is the training label used to fit the model, which we see during testing.

$\tilde{Y}_i$ is a new independent label for the same X_i . I.e. a fresh new outcome, for the same input X_i , like a test or generalization outcome.

Essentially, for the same input X_i the outcome Y might not be always identical because there is noise/randomness.

For instance, in the training data a student studied for 5 hours and got 80%. Y_i .

But in testing data, an unseen data point, another student studied 5 hours but got 85%. $\tilde{Y}_i$

Doing well on Y_i , the training outcome, does not guarantee doing equally well on $\tilde{Y}_i$, a fresh unseen outcome.

Back to the slide, the model's prediction is written as $\hat{f}(X_i)$.

The slide compares $(\tilde{Y}_i - \hat{f}(X_i))^2$ with $(Y_i - \hat{f}(X_i))^2$

The first one is like generalisation error, how well does the model do on a fresh outcome, then the second one is the training error, how well the model does on the exact label it was trained on.

The main result is that the expected gap between these two errors is: $(2/n) \sum_{i=1}^n \text{Cov}(Y_i, \hat{f}(X_i))$

The important part is the covariance, measuring how strongly the fitted prediction depends on the training label. If the model is very flexible, then $\hat{f}(X_i)$ is strongly influenced by Y_i . So the covariance is large, and the training error becomes very optimistic.

So, training error usually underestimates test error, especially when the model is very flexible and the fitted predictions depend strongly on the training labels.

Degrees of freedom measure how flexible a fitted model is. Higher degrees of freedom mean more flexibility. A more flexible model can adapt more strongly to the training data.

Consider the non-parametric regression model for (X, Y, U) , where

$$Y = f(X) + U,$$

where $Y, U \in \mathbb{R}$ and $X \in \mathbb{R}^d$ and $f : \mathbb{R}^d \rightarrow \mathbb{R}$. Assume

- ❶ $E[U|X] = 0$,
- ❷ $E[U^2|X = x] = \sigma^2$.

Consider a dataset $(X_i, Y_i, U_i)_{i=1, \dots, n}$. Then the degrees of freedom of an estimated function $\hat{f}$, conditional on features $X_1, \dots, X_n$, is defined as

$$df(\hat{f} | X_1, \dots, X_n) = \frac{1}{\sigma^2} \sum_{i=1}^n \text{Cov}(Y_i, \hat{f}(X_i)).$$

A non-parametric model is one that does not assume a fixed, small formula for the relationship between X and Y .

A parametric model would assume a specific form for $f(X)$, for example a straight line: $Y = \beta_0 + \beta_1 X + U$

A non-parametric model does not assume that $f(X)$ has to be a straight line, quadratic, or any fixed formula. It lets the data determine the shape of $f(X)$ more flexibly.

KNN is an example of a non-parametric method, simply uses nearby points.

Assumption 1 means the noise has average zero once we know x . So the model is not systematically too high or low.

Assumption 2 means the noise variance is constant and equal to σ^2 .

Then we define the degrees of freedom of the estimated function $\hat{f}$ as:

$$df(\hat{f} | X_1, \dots, X_n) = (1/\sigma^2) \sum_{i=1}^n \text{Cov}(Y_i, \hat{f}(X_i))$$

df means degrees of freedom. So df of the fitted model $\hat{f}$, given the feature values $X_1, \dots, X_n$

Again the key part is the covariance inside the sum.

This measures how much the fitted prediction at point X_i depends on the training label Y_i .

In other words, if changing Y_i would strongly change the prediction $\hat{f}(X_i)$, then the model has high degrees of

freedom.

Errors

We care about test error but we usually only observe training data.
So we can start with training error and then add a correction since it is usually too optimistic.
estimated test error = training error + correction

Examples are Mallows's C_p , AIC, BIC

The advantages are that they are mathematically justified and efficient.

The disadvantages are that the correction is not universal and it only works nicely when the model and loss function have a known structure.

The above examples work nicely in linear regression with a squared error but for more complicated models like KNN, decision trees, random forests, NNs, these models may not have a simple formula for the correction term. So it's not very flexible.

Instead, with an algorithmic approach, we directly estimate the test error by splitting the data, eg something like 50% training set, 25% validation, 25% test.

The model is fitted on the training set, validation set tunes parameters, such as k in KNN, then test set to estimate final performance on unseen data.

It is intuitive and works for many models and losses. Downside is that some data is held back from improving the model for validation and testing.

In the above split, we only use 50% of the data to train the model.

Cross-validation fixes this by rotating which part of the data is used for validation. Eg split the data into 5 equal parts (folds), train the model on 4, validate on the remaining 1. Repeat 5 times each time using a different fold as the validation set. Average the five validation errors.

Theorem (Stone, 1977)

If $k \rightarrow \infty$ and $k/n \rightarrow 0$, then for all distributions, the probability of error of the k -nearest neighbor classifier converges to the Bayes risk.

n number of training observations, k neighbours used for prediction, and where Bayes risk is the lowest possible classification error any classifier could achieve, even with perfect knowledge.

As the dataset gets larger ($k \rightarrow \infty$), should use more neighbours, do not keep k fixed because using less neighbours keeps the prediction more noisy. But we also require $k/n \rightarrow 0$ meaning k grows much more slowly than the total sample size n so that the KNN stays close.

Overall, if the dataset becomes very large, and we choose k carefully, k -NN classification can get as good as the best possible classifier.

Theorem (Cover and Hart, 1967)

The probability of error of 1-nearest neighbor $E[L_n]$ is for all joint distributions (X, Y) in the limit bounded by twice the Bayes risk, i.e.

$$\limsup_{n \rightarrow \infty} E[L_n] \leq 2B^*$$

For $K=1$, so 1-NN, the theorem is as the amount of data becomes very large, the error 1-NN is at most twice the Bayes risk, B^* , which is the best possible error rate.

L_n is the error rate of the 1-NN classifier with n training points. So $E[L_n]$ is the expected error rate.

Compared with the previous slide, KNN with chosen increasing K can converge to Bayes risk but 1NN at least won't reach beyond twice Bayes risk, so it can be noisy but theoretically not arbitrarily bad.

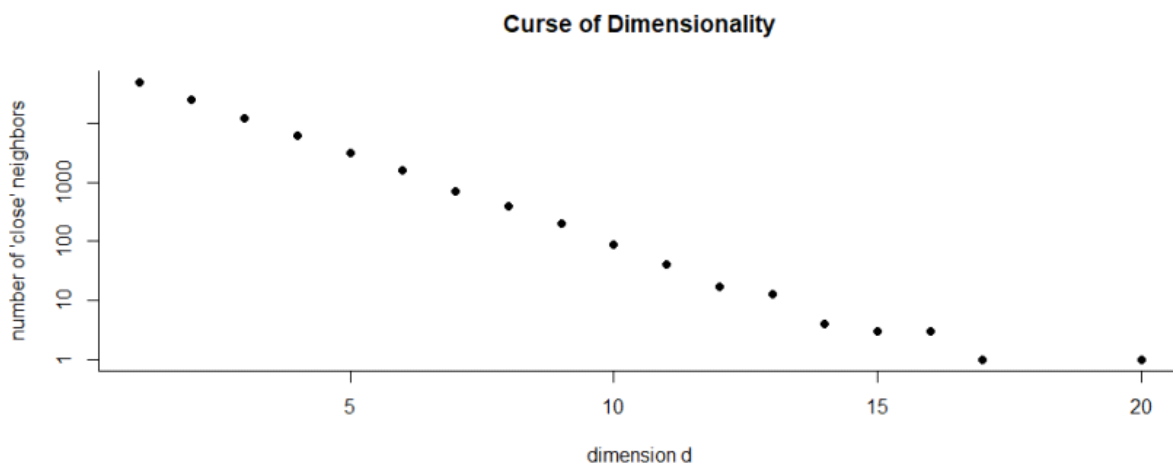
Gauß-Markov Theorem for OLS. We already know this, it is just that for a linear regression, OLS is BLUE (Best Linear Unbiased Estimator).

The assumptions are just that $E[U_i | X_i] = 0$, $E[U_i^2 | X_i] = \sigma^2$

Meaning the error has average zero once we know the features, and the error variance is constant (homoskedasticity).

There is also a rank condition, meaning there is enough variation in the predictors to estimate the coefficients properly, back to the aspect of the variance of beta (estimator) decreases when the variance of x increases.

Curse of dimensionality:



As the number of features/dimensions d increases, it becomes harder to find training points that are 'close' to a new test point.

Note that dimensions/features means the number of pieces of information used to describe each data point.

Eg for a house you can have features $X_1, \dots, X_4$ like size, beds, baths, age.

Four features, so four dimensions, but the more dimensions, the harder to find a close neighbour.

k -NN depends on finding nearby points. But when there are many features, "nearby" becomes rare.

Note: If you have no features, your best prediction is just the average of the observed outcomes.

Eg if we know nothing about a particular student and need to predict their exam result.

Do this if $d=1$ or $d=2$, but if $d \geq 3$, then you can apply a shrinkage, towards zero, which does improve the result.

$$\hat{\mu}^{\text{positive part James Stein}} = \left(1 - \frac{d-2}{n \sum_{j=1}^d (\hat{\mu}_j^{\text{mean}})^2} \right)_+ \frac{1}{n} \sum_{i=1}^n Y_i,$$

For a non-parametric model, like before, $Y = f(X) + U$ and the same two assumptions.

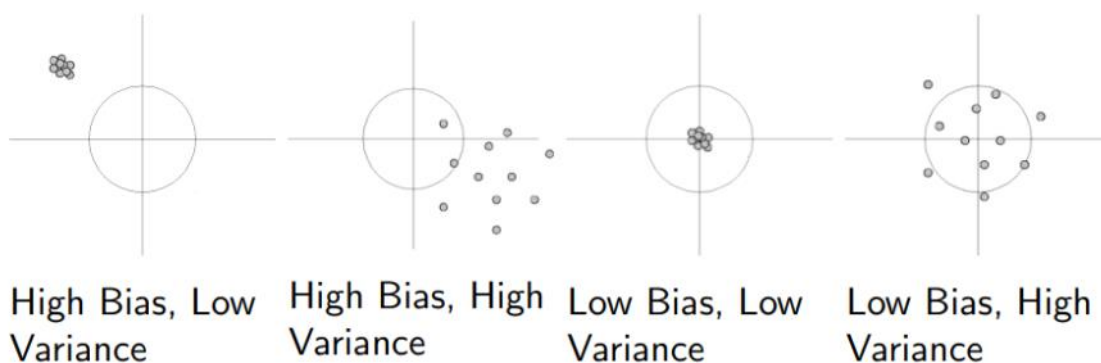
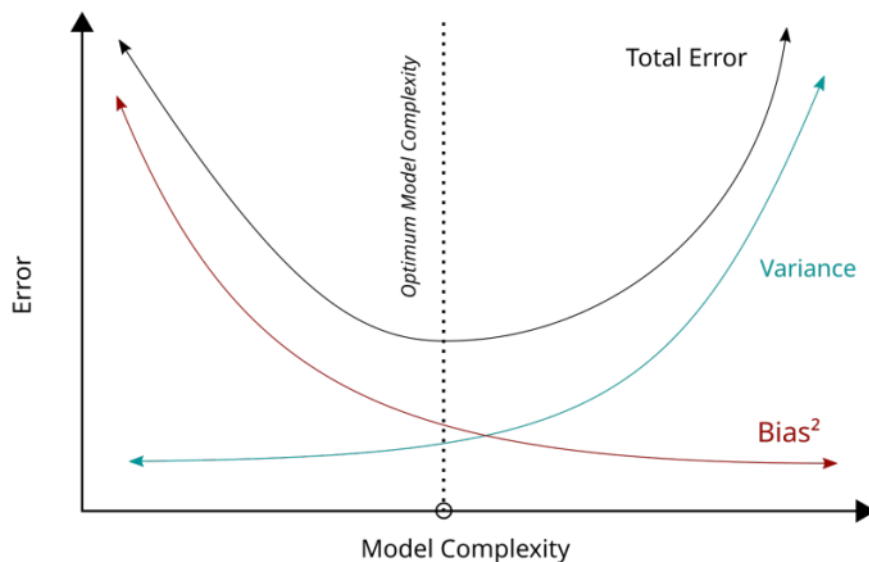
The MSE on an unseen datapoint (X^*, Y^*, U^*) can be broken down into three parts:

$MSE(\hat{f}|X^*) := \mathbb{E}[(\hat{f}(X^*) - Y^*)^2|X^*]$ into bias, variance, and irreducible error:

$$MSE(\hat{f}|X^*) = \left(\mathbb{E}[\hat{f}(X^*)|X^*] - f(X^*) \right)^2 + \mathbb{E} \left[\left(\hat{f}(X^*) - \mathbb{E}[\hat{f}(X^*)|X^*] \right)^2 \middle| X^* \right] + \sigma^2.$$

Bias² is how far off our model's average prediction is from the true $f(X)$. High bias = underfitting
 The second term is just the definition of variance, the sum of the difference between each term and the mean.
 How much our predictions fluctuate, high variance = overfitting
 σ^2 is noise inherent in the data that no model can ever remove

This formalises the overfitting/underfitting trade-off, a very flexible model has low bias but high variance, a rigid model has high bias but low variance, need to find the sweet spot.



 No free lunch theorem:

Averaged over all possible distributions of (X, Y) , every classification algorithm has the same error rate when classifying previously unobserved points.

But do we care about universally?

The above just means that no ML algorithm is universally better than any other. If an algorithm performs well on some problems, it must be performing worse on others to compensate.

Overall, don't look for one perfect algorithm, focus on matching the right algorithm to your specific problem and

data distribution.

Inference

09 June 2026 00:49

Starting out with a reminder on the CLT.

Theorem

For any $n \in \mathbb{N}$, let $X_1, \dots, X_n$ be i.i.d. draws with mean μ and variance $\sigma^2 < \infty$. Then as $n \rightarrow \infty$

$$\frac{1}{\sqrt{n}} \sum_{i=1}^n \frac{(X_i - \mu)}{\sigma} \xrightarrow{d} \mathcal{N}(0, 1),$$

No matter what the original distribution of X is, averaging enough of them together looks like a normal distribution.

Slutsky's Lemma

Theorem

For any $n \in \mathbb{N}$, let $X_1, \dots, X_n$ be a sequence of random variables and $Y_1, \dots, Y_n$ be a sequence of random variables. Assume that for some random variables X and a constant c ,

- $X_i \xrightarrow{d} X$ and
- $Y_i \xrightarrow{P} c$.

Then

- ① $X_i + Y_i \xrightarrow{d} X + c$ and
 - ② $X_i Y_i \xrightarrow{d} Xc$.
-

First, convergence in distribution means the shape of the distribution approaches (something).

Convergence in probability means the random variable essentially becomes a fixed number as n grows.

Generally, it means that as the sample grows, the probability of them deviating from a target by more than an arbitrarily small amount shrinks to zero.

The above matters because in inference, it allows us to estimate σ^2 when we don't know it.

With a standard linear regression with the standard assumptions:

$$\sqrt{n} \left(\hat{\beta}^{\text{OLS}} - \beta \right) \rightarrow \mathcal{N} \left(0, \sigma^2 \mathbb{E}[x_i x_i']^{-1} \right)$$

- if also $u_i \sim \mathcal{N}(0, \sigma^2)$, then

$$\sqrt{n} \left(\hat{\beta}^{\text{OLS}} - \beta \right) | X \sim \mathcal{N} \left(0, \sigma^2 \left(\frac{1}{n} X'X \right)^{-1} \right)$$

Result 1 means our OLS estimate is not exactly equal to the true parameter, but we can quantify the uncertainty and it follows a normal distribution. The variance term $\sigma^2 \mathbb{E}[x_i x_i']^{-1}$ tells us how spread out our estimates are

Result 2 is stronger but requires the stricter assumption that errors are actually normally distributed.

Regression with one confounder

Suppose we have: $y_i = d_i\alpha + x_i\beta + \varepsilon_i$

We only care about estimating α and β is a nuisance parameter, we don't care about it.

So do we include x in the regression (too be discussed, setting up notation below)?

This question matters because if we include X_i when $\beta = 0$ (it has no real effect), we waste statistical power and our confidence intervals for α get wider unnecessarily.

If we exclude x_i when $\beta \neq 0$ (it does have an effect), we get omitted variable bias — our estimate of α is systematically wrong.

$M_0(\alpha, \beta)$ as the "most parsimonious model", meaning the simplest correct model:

$$M_0(\alpha, \beta) = \begin{cases} U = \text{Unrestricted} & \text{if } \beta \neq 0, \\ R = \text{Restricted} & \text{if } \beta = 0. \end{cases}$$

If $\beta \neq 0 \rightarrow$ the true model is Unrestricted (U) — x belongs in the regression

If $\beta = 0 \rightarrow$ the true model is Restricted (R) — x has no effect, leave it out

Just setting up notation that there is a model M_0 which we don't know nor do we know whether β is really zero or not.

$$\hat{M}(\hat{\alpha}(U), \hat{\beta}(U)) = \begin{cases} U = \text{Unrestricted} & \text{if } \sqrt{n} \left| \frac{\hat{\beta}(U)}{\sigma_{\beta,n}} \right| > c, \\ R = \text{Restricted} & \text{if } \sqrt{n} \left| \frac{\hat{\beta}(U)}{\sigma_{\beta,n}} \right| \leq c. \end{cases}$$

Since we don't know it, we estimate it using a threshold c .

If our standardised estimate of β is large enough, we pick Unrestricted, otherwise Restricted. Basically a hypothesis test asking if it is statistically different from 0.

$$\hat{\alpha}(R) - \alpha = \frac{\overline{\text{Cov}(d, x)}}{\overline{\text{Var}(d)}} \beta.$$

This is the omitted variable bias formula.

$\hat{\alpha}(R) - \alpha$ is simply how wrong your estimate of α is when you exclude x from the regression.

RHS says this is equal to the covariance of the treatment variable d and the confounder x . If uncorrelated/no covariance, then no bias an excluding x is fine. If highly correlated, bias is large.

$\text{Var}(d)$ — just a scaling factor.

Then β is how strongly x affects y . If $=0$, no effect on y , excluding it also causes no bias, vice versa.

The upside in model selection is that we get tighter intervals, the downsides of getting the model wrong is systematic bias. So can we reliably enough identify the correct model for this to be worthwhile?

We want consistency:

$$P[\hat{M} = U \mid U] \rightarrow 1$$

$$P[\hat{M} = R \mid R] \rightarrow 1$$

The above mean picking the correct model with probability 1.

Taking the first one, with some maths, plug in the result (in $\beta = 0$ (i.e. the true model is restricted)) of rearranging and by $c \rightarrow \infty$ makes this $\rightarrow 0$.

I.e. if we make the threshold c very large, we are making it very hard to select the unrestricted model.

So if the true model is actually restricted ($\beta = 0$), the signal in the data is weak/non-existent, and with a high threshold c we'd almost never incorrectly declare it unrestricted.

C is like the burden of proof and setting that bar very high when a genuinely zero β will essentially never clear it.

Alternatively, if we plug it into $\beta \neq 0$ (true model is unrestricted), that shows letting $c = c_n$ grow but slower than $\sqrt{n}$ (i.e. $c_n/\sqrt{n} \rightarrow 0$) ensures this $\rightarrow 1$.

So now we have a c that satisfies both conditions:

$$c_n \rightarrow \infty \text{ ensures } P[M = U \mid R] \rightarrow 0$$

$$c_n/\sqrt{n} \rightarrow 0 \text{ ensures } P[M = U \mid U] \rightarrow 1$$

But, we will see that even though we've achieved model selection consistency, our inference on α can still be wrong. Getting the model right asymptotically doesn't automatically mean your confidence intervals for α are valid.

Note, asymptotically means 'as the sample size n grows towards infinity'.

Since $\hat{M} \rightarrow M$ (we select the right model asymptotically), our estimate $\hat{\alpha}(\hat{M}) \rightarrow \hat{\alpha}(M)$, where $\hat{\alpha}(M)$ is called the oracle estimator — the estimator that magically knows the true model.

This model has the same asymptotic distribution $\hat{\alpha}(M)$ for local deviations of α and $\beta \rightarrow$ inference works

But not uniformly over all α and $\beta \rightarrow$ inference can fail even asymptotically.

In simpler words and taking a slight step back, we have d and x , and we are not sure whether to include x in the regression. Include is unrestricted and exclude is restricted.

We have seen with the right threshold, our model selection gets it right as n grows. Now we are saying that after selecting our model, we want to make confidence bounds like saying α is between 2 and 5 with 95% confidence. This is inference.

After all, inference is the process of using a trained model to make predictions, draw conclusions, or generate new content based on new, previously unseen data.

The naïve approach would be to pick a model, then just do normal OLS inference as if we've always known that was the correct model. The issue is that when β is very small (not zero, but tiny and barely detectable) sometimes it picks restricted and sometimes unrestricted. The intervals end up being too narrow.

This is the core Leeb and Pötscher warning — model selection and inference cannot simply be treated as two separate independent steps.

For estimating α yes it doesn't really matter since β is very small, but for inference, confidence intervals, it can be an issue if they end up too narrow. So the bias itself might be negligible but the overconfidence in the results are not.

Imagine $\beta = \beta_n = t/\sqrt{n}$ — a beta that shrinks towards zero as n grows.

t is some constant but n grows as sample size grows, and so β_n shrinks to zero.

From CLT we know: $\sqrt{n}(\hat{\beta} - \beta) \rightarrow N(0, \sigma^2)$

Rearranging this, the estimation error is: $\hat{\beta} - \beta \approx N(0, \sigma^2/n)$

So the standard deviation of your estimation error is approximately $\sigma/\sqrt{n}$.

As n grows, your estimation error shrinks at rate $1/\sqrt{n}$. For example:

$n = 100 \rightarrow$ error is roughly $\sigma/10$

$n = 10000 \rightarrow$ error is roughly $\sigma/100$

$\beta = t/\sqrt{n}$ causes problems because estimation error and β are shrinking at exactly the same rate, it's like a race where you're always the same distance behind. With more data you are getting better at detecting smaller β but this is getting smaller at the exact same speed. So the size of the signal β and the noise (estimation error) remain the same size relatively.

This regime represents the worst case scenario for model selection. β is technically non-zero so the unrestricted model is correct, but it's so small your test keeps missing it and picking restricted instead.

$$P[\hat{M} = U \mid U] = 1 - 1 + 0 = 0$$

Meaning we almost never select the correct model.

This is the formal proof of why naive inference fails. There will always exist values of β — specifically those around $t/\sqrt{n}$ — where your model selection goes wrong often enough to corrupt your confidence intervals for α , no matter how large n gets.

The impact on inference, what the previous slides are saying:

- How bad is such a deviation?
 - no effects for estimating α : $\hat{\alpha}_n(\hat{M}) \rightarrow \alpha$
 - still have $\sqrt{n}(\hat{\alpha}_n(\hat{M}) - \alpha)$ converging to a non-degenerate distribution
 - but the distribution of $\sqrt{n}(\hat{\alpha}_n(\hat{M}) - \alpha)$ need not be normal and the asymptotic variance may be larger than the naive inference would suggest

So how bad is this deviation?

Point 1 says our estimation of α is fine

Point 2 says it still converges to a non-degenerate distribution

A degenerate solution is when a random variable converges to a single fixed number with probability 1. There's no uncertainty left at all.

For example if I told you $\sqrt{n}(\hat{\alpha} - \alpha) \rightarrow 5$, that would be degenerate — it's just a constant, not a distribution.

So non-degenerate means the error $\sqrt{n}(\hat{\alpha}_n(\hat{M}) - \alpha)$ still has genuine randomness and spread around it — it hasn't collapsed to a single point.

After reading point 3 you will see that point 2 means that yes the distribution is wrong (not normal) and the variance is bigger than you think but at least it hasn't broken down into something useless.

Point 3 says the issue. Our distribution is not necessarily normal, so using normal distribution based confidence intervals is wrong. It has larger variance than we think (than what our naive inference accounts for) so our confidence intervals are too narrow.

This issue never goes away despite how much data we have.

- $OVB = \frac{\overline{\text{Cov}}(d, x)}{\overline{\text{Var}}(d)} \beta$

- if $\beta \sim 1/\sqrt{n}$, then $OVB \sim 1/\sqrt{n}$

- as seen before, this is enough to invalidate naive inference

Omitted Variable Bias (OVB), from before.

But one observation, if $\beta \sim 1/\sqrt{n}$, then $\text{OVB} \sim 1/\sqrt{n}$ and that this is enough to invalidate naïve inference.

We said before that if β is tiny, the bias in α is tiny so it doesn't matter much for estimation, that's still true but the issue is $1/\sqrt{n}$ is exactly the rate at which estimation error shrinks. So an OVB of size $1/\sqrt{n}$ is not negligible relative to your estimation error (they are the same) so bias would be as large as estimation error.

It's like measuring something with a ruler, with 1mm error in measurement, that's the limit of precision.

Double machine learning:

New model selection rule that uses two thresholds instead of one:

- double ML model selection

$$\hat{M}^{dML}(\hat{\alpha}(U), \hat{\beta}(U)) = \begin{cases} U & \text{if } \sqrt{n} \left| \frac{\hat{\beta}(U)}{\sigma_{\beta,n}} \right| > c_1 \text{ or } \left| \widehat{\text{Cov}}(d, x) \right| > c_2, \\ R & \text{if } \sqrt{n} \left| \frac{\hat{\beta}(U)}{\sigma_{\beta,n}} \right| \leq c_1 \text{ and } \left| \widehat{\text{Cov}}(d, x) \right| \leq c_2. \end{cases}$$

Choose Unrestricted if either $\sqrt{n} |\hat{\beta}(U)/\sigma_{\beta,n}| > c_1$ or $|\widehat{\text{Cov}}(d, x)| > c_2$

Choose Restricted only if both are below their thresholds

We used to run one test which asked "is β big enough to matter?" If yes include x , if no exclude it.

The problem was: you could be just wrong enough about β to cause inference problems.

OVB causes problems if both:

β is non-negligible (x actually affects salary)

$\text{Cov}(d, x)$ is non-negligible (education and training are correlated)

So run two tests instead, asking if β is big enough and are d, x correlated. Only exclude x if both tests say it is safe enough to do so.

To incorrectly choose restricted would be to both miss that β is large enough and miss that $\text{Cov}(d, x)$ is large enough.

The probability of both errors happening together is much smaller — order $1/n$ rather than $1/\sqrt{n}$ — which is small enough that it no longer corrupts the confidence intervals and does not cause issues with inference.

=====

Example through made up numbers, feel free to skip:

Trying to measure the effect of a job training program on salary.

y is salary

d is if they did the training

x is years of education

The true model is: $y = 10000d + 500x + u$

So the true effect of training is $\alpha = £10,000$ and education has effect $\beta = £500$ per year.

Should we include x ? Education and training are correlated -- $\text{Cov}(d, x) \neq 0$.

But $\beta = 500$ is fairly small. You run a test and it comes back borderline — not clearly significant. A test says that β is not significant and you should not include x .

So you run: $y = \alpha \cdot d + u$; You get $\hat{\alpha} = £11,800$ instead of the true $£10,000$.

The OVB = Cov(d,x)/Var(d) · 500 pushed your estimate up by £1,800.

You then construct a 95% confidence interval: £10,500 to £13,100. The true parameter value £10000 is not even in the interval, too narrow and in the wrong place, exactly the Leeb and Pötscher problem.

Instead you run two tests: Is β large enough (not quite) and is Cov(d,x) large enough (yes)?

As at least one test says yes, you include x.

$$y = \alpha \cdot d + \beta \cdot x + u$$

You get $\hat{\alpha} = £10,200$ — much closer to the truth.

Your 95% confidence interval: £9,100 to £11,300

The true value £10,000 is comfortably inside the interval

=====

The below is theoretical, in the sense that if we knew the true model, parameters, error distribution and Cov(d,x). Instead everything must be estimated from data. The theoretical analysis does, though, show us the quantities we need to be careful for.

- suppose $\overline{d_i} = 0$, $\overline{d_i^2} = 1$, $\overline{x_i} = 0$
- case 1: x included
 - inference valid although possibly overly conservative
- case 2: x excluded
 - OVB logic

$$\begin{aligned}\hat{\alpha} &= \overline{d_i y_i} = \overline{d_i(d_i \alpha + x_i \beta + u_i)} \\ &= \alpha + \overline{d_i x_i} \beta + \overline{d_i u_i}\end{aligned}$$

- then

$$\begin{aligned}\sqrt{n}(\hat{\alpha} - \alpha) &= \sqrt{n} \overline{d_i x_i} \beta + \sqrt{n} \overline{d_i u_i} \\ &= \sqrt{n} \frac{1}{n} \sum_{i=1}^n d_i x_i \beta + \sqrt{n} \frac{1}{n} \sum_{i=1}^n d_i u_i \\ &= \sqrt{n} \widehat{\text{Cov}}(d, x) \beta + \frac{1}{\sqrt{n}} \sum_{i=1}^n d_i u_i\end{aligned}$$

This just works through two cases, x included or not.

In Case 1 where x is included, the inference is valid but possible overly conservative and the confidence intervals might be slightly wider than necessary but will contain the truth.

Case 2 is where x is excluded.

Remember that for a parameter, the OLS best estimator is Cov(x,y)/Var(x)

Using the assumptions, it winds up being through plugging into the above that the OLS estimate of α (when x is excluded) is the average of d times y. Then sub in the true model $y = \alpha + x\beta + u$.

Error is our estimate minus the real, multiply by $\sqrt{n}$ because:

The CLT says for i.i.d. draws with mean μ and variance σ^2 :

$$(1/\sqrt{n})\sum(X_i - \mu) \rightarrow N(0, \sigma^2) \text{ -- alternatively written as } \sqrt{n}(\bar{X} - \mu) \rightarrow N(0, \sigma^2)$$

Meaning the raw error $\bar{X} - \mu$ shrinks at rate $1/\sqrt{n}$ as n grows. So if you multiply it back by $\sqrt{n}$ you get something that stabilises — it doesn't shrink to zero or blow up to infinity, it settles into a nice normal distribution.

If this term converges to a known normal distribution, we can build valid confidence intervals around it.

$\sqrt{n}$ is essentially the perfect scaling factor that catches the error term at just the right moment — after it has shrunk enough to be well behaved but before it disappears completely to zero.

Back to the slide, plug that in, rewrite the average in summations, simplify.
Now the final line is what is interesting.

$\sqrt{n} \cdot \text{Cov}(d, x) \beta$ — this is the dangerous OVB term. Notice it has $\sqrt{n}$ out front meaning it can grow with sample size rather than shrink — that's what makes it problematic for inference.

Under double ML this becomes small enough to not cause inference problems. **

$(1/\sqrt{n}) \sum d_i u_i$ — this is just noise. By the CLT this converges to a normal distribution as n grows — well behaved and expected.

So through this theoretical analysis, we see that the estimated quantities to look out for are OVB and to use the double ML approach to check both estimates.

Back to **

- critical term is $\sqrt{n} \widehat{\text{Cov}}(d, x) \beta$

- if we excluded x , then

- $|\hat{\beta}| \leq \frac{c_1}{\sqrt{n}}$

- $|\widehat{\text{Cov}}(d, x)| \leq \frac{c_2}{\sqrt{n}}$

- worst case before: $|\beta| = \frac{c_3}{\sqrt{n}}$

- then

$$|\sqrt{n} \widehat{\text{Cov}}(d, x) \beta| \leq \sqrt{n} |\widehat{\text{Cov}}(d, x)| |\beta| \leq \sqrt{n} \frac{c_2}{\sqrt{n}} \frac{c_3}{\sqrt{n}} = \frac{c_2 c_3}{\sqrt{n}}.$$

- uniformly valid oracle inference!

Critical term is the OVB.

If we incorrectly excluded x , double ML thresholds mean we must have both:

$|\hat{\beta}| \leq c_1/\sqrt{n}$ — our estimate of β was small enough to pass threshold 1

$|\text{Cov}(d, x)| \leq c_2/\sqrt{n}$ — our estimate of covariance was small enough to pass threshold 2

Bounding through what we know is larger than or equal to and then rearranging.

What this means: uniformly valid oracle inference on the bottom line.

Oracle inference from before. Oracle estimator is the estimator that theoretically knows the true model. Oracle inference is the inference you would get if you always knew whether to include x or not.

Uniformly valid is the key word from the Leeb and Pötscher problem. This is from before and means the inference works correctly for all values of α and β , not just most of them.

The statement follows directly from the final bound: $|\sqrt{n} \cdot \text{Cov}(d, x) \cdot \beta| \leq c_2 c_3 / \sqrt{n}$

Where this goes to zero as n grows, regardless of what α and β actually are. There's no special value of β that makes this blow up, unlike the naive single threshold case.

Overall:

Your confidence intervals are as good as if you knew the true model (oracle)

This works for all possible values of α and β (uniformly)

Not just asymptotically for most values, but genuinely robustly (valid)

The whole point of double ML: by requiring two conditions to both be satisfied before excluding x , you force the OVB into a regime where it's too small to matter for inference.

Final slide, partially linear model, just an introduction:

$$Y_i = D_i\alpha + g(X_i) + U_i$$

$$D_i = m(X_i) + V_i$$

Where g and m can now be any function, not just simple linear $x\beta$. Now we are saying that x can affect y in any complicated nonlinear way.

The solution is double LASSO to select which polynomials matter in either regression. This generalises double ML into a high dimensional setting.

The difference is now instead of one nuisance parameter β you potentially have many polynomial terms to deal with.

Because the model is partly linear — $D_i\alpha$ is linear in D , but $g(X_i)$ can be any nonlinear function. So it sits between a fully linear model and a fully nonlinear one.

Regularisation

09 June 2026 00:49

Let's say we have:

- $\min_{\beta} \frac{1}{2} \frac{1}{n} \sum_{i=1}^n (Y_i - X_i' \beta)^2$
- FOC $\frac{1}{2} \frac{1}{n} \sum_{i=1}^n X_i (Y_i - X_i' \hat{\beta}^{\text{OLS}}) = 0$
- solution
 - assume $\frac{1}{n} \sum_{i=1}^n X_i X_i'$ is invertible
 - $\hat{\beta}^{\text{OLS}} = \left(\frac{1}{n} \sum_{i=1}^n X_i X_i' \right)^{-1} \frac{1}{n} \sum_{i=1}^n X_i Y_i$
 - in matrix notation $\hat{\beta}^{\text{OLS}} = (X'X)^{-1} X'Y$

We want to find the β that minimises the top expression, do first order condition.
We are using 'real' data so min the average error over actual data.

Note about when to use $1/(n-1)$:

$n-1$ (Bessel's) correction is specifically for getting an unbiased estimate of the population variance σ^2 .

After you have found $\hat{\beta}_{\text{OLS}}$

If you want to estimate the noise variance (epsilon), you'd use: $\hat{\sigma}^2 = \text{SSR} / (n - p)$

Back to the slide:

Then the rest is rearranging.

We will come to find out: the critical assumption is that $(X'X)$ is invertible. This is fine when you have more observations than features, but this breaks in high dimensions (when $p > n$) which is why we need regularisation.

Where n is number of observations (rows), p number of features/regressors (columns).

We are building up towards OLS for ml/prediction after that OLS recap.

Can we use OLS to approximate *any* function?

The theorem says:

For any continuous function f on an interval $[a,b]$, you can approximate it arbitrarily closely using a polynomial:

$$\sup_{x \in [a,b]} \left| f(x) - \sum_{k=0}^K a_k x^k \right| \leq \varepsilon.$$

The error can be made as small as you like by choosing K large enough.

A function that goes up and down many times is wiggly, the more wiggly f is, the higher K has to be to capture all of those turns. Higher k can capture more flexible functions -> less bias. But, higher K means more features and model clings too closely to the training data -> higher variance (below). This is the bias-variance trade-off.

K is the power/polynomial term and it means more features going into OLS, pushing towards the $p > n$ problem which we'll see and risk overfitting.

So even though Weierstraß guarantees we can approximate any function, doing so in practice comes at a cost.

OLS for Nonparametric Regression

If $E[Y|X=x] = f(x)$ is continuous on $[a,b]$ (we have to keep saying that the function is bounded), we can approximate it by regressing Y on polynomial features up to degree K :

$$\hat{f}(x) = \sum_{k=0}^K \hat{a}_k^{OLS} x^k$$

Degree k is a tuning parameter. Degrees of freedom $K+1$

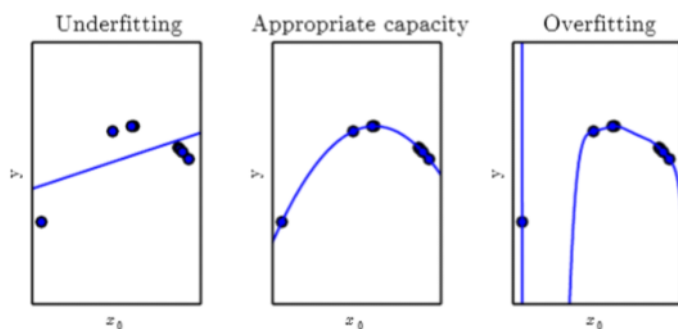


Figure 5.3 of "Deep Learning"

Underfitting, just right, overfitting.

OLS in High Dimensions:

Two regimes, the classical regime where p is fixed, $n \rightarrow \infty$. OLS works well and gives consistency.

High dimensional regime, new problem.

$p \rightarrow \infty$, $n \rightarrow \infty$, and $p/n \rightarrow \infty$

i.e. the number of features grows with the data, and can exceed it

What goes wrong when $p > n$:

$X'X$ is no longer invertible, which means $\hat{\beta} = (X'X)^{-1}X'Y$ breaks down.

Specifically:

The smallest eigenvalue of $X'X$ becomes zero (recap below).

There exists a $\beta \neq 0$ such that $X\beta = 0$

This causes numerical instability

Hence the fundamental problem is that if we have more features than observations (common in ML) OLS cannot be computed in the usual way.

Eigenvector recap:

A matrix like $X'X$ is a transformation that when applied stretches or shrinks that matrix in different directions.

An eigenvector is a special direction where the matrix only stretches/shrinks, doesn't rotate.

An eigenvalue is how much it stretches/shrinks in that direction.

If an eigenvalue = 5, it means $X'X$ stretches vectors in that direction by a factor of 5.

If an eigenvalue = 0, it means $X'X$ completely collapses vectors in that direction to zero — i.e. information is lost.

To compute OLS, we need to invert $X'X$. Inverting involves dividing by eigenvalues, so small eigenvalue explodes it, unstable estimates. Zero eigenvalue $\rightarrow$ dividing by zero $\rightarrow$ cannot invert at all

The slide information above said that the smallest eigenvalue of $X'X$ becomes zero when $p > n$. This means $X'X$

cannot be inverted, OLS breaks down completely.

Recap done.

There are three approaches to save OLS in high dimensions

Numerical stabilisation of the inverse

Early stopping

Bayes (statistics)

Mathematical groundwork for solution 1:

We can decompose $(1/n)X'X$:

$$(1/n)X'X = Q\Lambda Q^t$$

Where:

Λ is a diagonal matrix with the eigenvalues along the diagonal

Q is a matrix of the corresponding eigenvectors, and is orthonormal (meaning $Q^tQ = QQ^t = I$)

Some rearranging later and the key result is:

Invertibility of $(1/n)X'X$ depends entirely on the eigenvalues in Λ .

$(1/n)X'X$ is invertible iff 0 is not an eigenvalue.

Stabilising the inverse, solution 1:

Even if the smallest eigenvalue of $(X'X/n)$ is zero, if we add a small positive number λ to the diagonal:

$$(X'X/n) + \lambda I$$

then the smallest eigenvalue is now $\lambda > 0$, which means it's always invertible. We just choose λ as small as possible to stabilise it.

What the actual slide says:

- **An early idea: stabilize the inverse**

- even if $0 = \min_{\beta \neq 0} \beta' \frac{X'X}{n} \beta$
- $0 < \min_{\beta \neq 0} \beta' \left(\frac{X'X}{n} + \lambda I_p \right) \beta$

$\beta'A\beta$ is called a quadratic form. It's a way of measuring how much a matrix A "stretches" a vector β . The result is just a single number (scalar).

Second line says there exists some direction β where $X'X/n$ completely collapses to nothing.

This confirms the smallest eigenvalue is zero $\rightarrow$ not invertible.

Third bullet says: By adding λI_p , we've guaranteed the minimum stretching is now at least $\lambda > 0$ in every direction $\rightarrow$ always invertible.

Progressing through the slide:

Back in the slides we had the original OLS estimator: $\hat{\beta} = (X'X/n)^{-1} \cdot X'y/n$

The problem is $(X'X/n)$ might not be invertible when $p > n$.

Instead of inverting $(X'X/n)$, we invert $(X'X/n + \lambda I)$ instead.

This gives us the Ridge estimator:

$$\hat{\beta}^R = \left(\frac{X'X}{n} + \lambda I \right)^{-1} \frac{X'y}{n}$$

This is in the regular OLS form of $(W'W)^{-1}W'y$

Ridge is just OLS but with x, y stacked with extra rows; the extra rows are $\sqrt{\lambda} \cdot I_p$ added to X , and zeros added to y .

$$W = \begin{pmatrix} X/\sqrt{n} \\ \sqrt{\lambda} I_p \end{pmatrix}, \quad v = \begin{pmatrix} y/\sqrt{n} \\ 0_p \end{pmatrix}$$

Ridge is like a penalised version of OLS:

$$\underbrace{\min_{\beta} \|v - W\beta\|_2^2}_{\text{RSS in adjusted OLS solved by Ridge}} = \min_{\beta} \underbrace{\frac{1}{n} \|y - X\beta\|_2^2}_{\text{RSS in original OLS}} + \underbrace{\lambda \|\beta\|_2^2}_{l_2 \text{ penalty}}$$

RSS is just SSR, sum of squared residuals.

Ridge = OLS RSS + a penalty on the size of β .

Expanding on the penalty part:

$\|\beta\|^2 = \beta_1^2 + \beta_2^2 + \beta_3^2 + \dots + \beta_p^2$ --- it's just the sum of squared coefficients. How large the coefficients are overall.

A very large β_1 might fit the data brilliantly (low RSS), but makes $\|\beta\|^2$ large, which gets penalised.

Large coefficients are a sign of overfitting, trying to assign all variance in the data to an estimator, which is not possible.

Overall, it means keep whatever parameters you have reasonably small.

LHS is just OLS.

First part of RHS is usual SSR, how well β fits the data

$\lambda \|\beta\|^2$ — this is the l_2 penalty, penalising large coefficients

The penalty form above and the constrained form from before give the same solution; for every value of λ there is a corresponding c that gives the same $\hat{\beta}^R$, and vice versa.

The extreme cases are:

$\lambda = 0$ (no penalty), penalty term vanishes and you are just doing pure OLS and beta can be as large as it wants.

$\lambda = \infty$ (infinite penalty), any $\beta \neq 0$ would incur an infinite penalty, so the only affordable solution is: $\beta^R = 0$

Model ignores the data completely.

Note that OLS is unbiased: $E[\hat{\beta}] = \beta$

But ridge is not. By shrinking coefficients you are deliberately pulling them away from their true values, so if a feature genuinely has a large true coefficient, ridge will underestimate it when pulling it closer to zero.

You accept this bias because OLS is unbiased but can have huge variance in high dimensions, whereas ridge is biased but has much lower variance in this case. Bias-variance trade-off.

A slightly wrong but stable estimate can actually outperform an unbiased but wildly variable one, especially on new data.

We will find why LASSO can be favourable to this (coming later).

Early stopping method (second solution):

Instead of solving OLS exactly, what if we use gradient descent (an iterative algorithm) to minimise the RSS, but stop early before it fully converges?

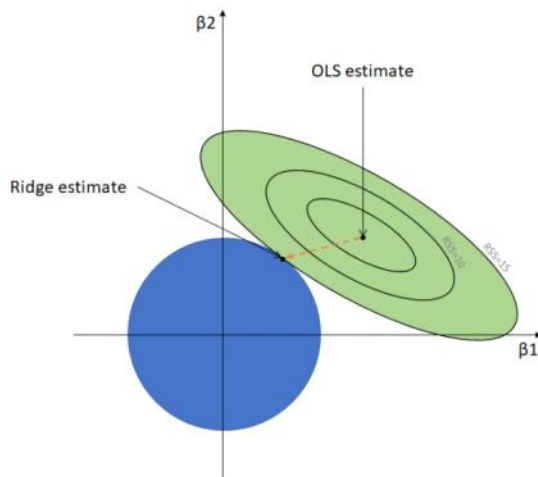


Figure taken from: "Ridge Regression for Better Usage" by datascience.eu

Green ellipses — contour lines of the RSS. The centre of the ellipses is β (the OLS estimate) — the point of lowest RSS.

Blue circle — the constraint region $||\beta||^2 \leq c$ i.e. where Ridge is allowed to look for solutions.

OLS with a perfect fit but potentially large coefficients would sit at the centre of the green ellipses.

Ridge must find the best solution within or on the blue circle. The ridge estimate is where the smallest possible ellipse touches the circle.

The ridge estimate is pulled away from the OLS towards the origin and how far this is depends on λ .

Larger λ means smaller circle, means more shrinkage.

Slide 19

OLS objective can be written as:

$$f(\beta) = \frac{1}{2}(\beta - \hat{\beta}^{\text{OLS}})' \frac{1}{n} X'X (\beta - \hat{\beta}^{\text{OLS}}) + f(\hat{\beta})$$

Then sets up gradient descent, starting from $\beta^0 = 0$, with step size α :

$$\beta^{k+1} = \beta^k - \alpha \nabla f(\beta^k)$$

kth iteration.

Rearrange and with the eigendecomposition

We then see that the gradient descent estimator after k steps is: $\beta^k = Q(I - (I - \alpha\Lambda)^k)Q^t\hat{\beta}$

Ridge estimator is: $\hat{\beta}^R = Q(I - (\Lambda + \lambda I)^{-1}\lambda)Q^t\hat{\beta}$

These two expressions have the same structure — both are scaled versions of β

They are equal when: $(I - \alpha\Lambda)^k = (\Lambda + \lambda I)^{-1}\lambda$

Using $\log(1+x) \approx x$, these are approximately equal when: $\lambda \approx 1/(k\alpha)$

Intuitively, more gradient descent steps (larger k) $\rightarrow$ smaller effective $\lambda \rightarrow$ less shrinkage

Fewer steps (smaller k) $\rightarrow$ larger effective $\lambda \rightarrow$ more shrinkage

So the number of steps k plays the same role as λ in Ridge — it's the tuning parameter controlling how much shrinkage you apply.

Overall: **Early stopping and Ridge regression are essentially doing the same thing** — they're just two different

ways of preventing OLS from overfitting in high dimensions.

Bayes statistics, third solution

Before, β was unknown, but fixed and we are trying to estimate it and use data to get as close as possible.

With Bayesian, β is treated as random (it has a probability distribution) and we start with a prior belief before seeing the data, then update it as we use it.

The update rule.

$$f(\beta|Y, X) = \frac{f(Y|X, \beta)f(\beta)}{f(Y)}$$

$f(\beta)$ is the prior — what we believe about β before seeing data

$f(Y|X, \beta)$ is the likelihood — how well β explains the data

$f(\beta|Y, X)$ is the posterior — our updated belief about β after seeing data

Works well if the prior can encode correct priori (knowledge/reasoning formed theoretically/intuitively prior to gathering real data) meaning if we know something about β before seeing data, you can build that in.

It also rules out weird cases; in high dimensions, OLS can produce wild and implausible results. A prior can say eg ' β is unlikely enormous' and can naturally pull towards more sensitive values.

But, what these 'weird' cases are subjective. Bayesian methods depend on the prior choices made and it's not objective like OLS. There's no single "correct" prior. Different priors lead to different posteriors and different estimates — so the result depends on a somewhat arbitrary choice.

A prior can also be wrong.

We will see that Ridge regression has a natural Bayesian interpretation.

Assume $Y|X, \beta \sim N(X\beta, \sigma^2 I_n)$ — the data is normally distributed around $X\beta$ with noise σ^2 ; and that

$\beta \sim N(0, \tau^2 I_p)$ — our prior on β is that it's normally distributed around zero

The second prior is saying before we see any data,

β is probably close to zero (large unlikely) and all coefficients are equally likely a priori.

This is like the shrinkage Ridge does.

Maximising the posterior likelihood (i.e. finding the most probable β given the data) solves:

$$\min_{\beta} (Y - X\beta)^t (Y - X\beta) + \frac{\sigma^2}{\tau^2} \|\beta\|_2^2$$

Which is exactly Ridge regression with $\lambda = \sigma^2/\tau^2$!

This gives us the following Ridge interpretation:

σ^2 is how noisy the data is

τ^2 is how large we think β is a priori

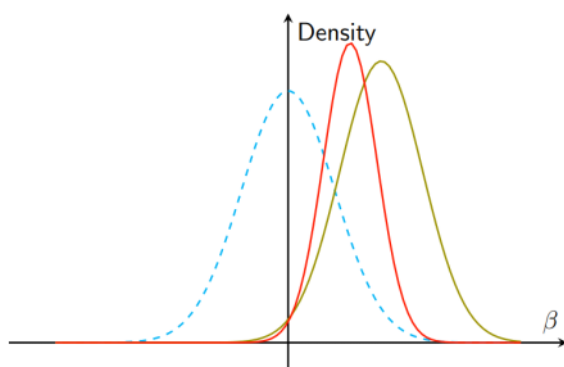
Noisier data $\rightarrow$ larger $\lambda \rightarrow$ more shrinkage

If we believe β is large (large τ^2) $\rightarrow$ smaller $\lambda \rightarrow$ less shrinkage

So Ridge is Bayesian OLS with a normal prior.

--

Bayesian Updating



--- Prior --- Likelihood --- Posterior

Prior is what we believe about the data before seeing any, likelihood is what the data alone suggests about β , peaked around the OLS estimate. Posterior is our updated belief after combining prior and data, somewhere in between, compromise between prior and what the data tells us (likelihood).

From before, our prior is $N(0, \tau^2 I_p)$ — centred at zero. Prior pulls β towards zero, the likelihood pulls β towards $\hat{\beta}$ (OLS) and the posterior (Ridge estimate) lands somewhere in between.

With less data, the prior dominates which makes sense because you should likely trust the prior when more data is not available.

Regularized regression

Subset selection then LASSO

Refresh that we have seen three ways of moving from OLS to Ridge (numerical stabilisation, early stopping, Bayesian prior), all roads led to Ridge.

Instead of just Ridge, we can write a general regularised regression problem:

$$\min_{\beta} \frac{1}{n} \|Y - X\beta\|_2^2 + \lambda P(\beta)$$

Which is just OLS + penalty term

Where $P(\beta)$ is any penalty term on the complexity of β . $\beta=0$ is low complexity. The idea is to penalise distance to low complexity.

$$P(\beta) = \begin{cases} \|\beta\|_2^2 & \text{Ridge} \\ \|\beta\|_1 & \text{LASSO} \\ \sum_{j=1}^p \mathbf{1}_{\beta_j \neq 0} =: \|\beta\|_0 & \text{Subset Selection} \\ \alpha \|\beta\|_1 + \frac{1-\alpha}{2} \|\beta\|_2^2 & \text{Elastic Net} \end{cases}$$

The above shows four examples of choice of penalty which we will see.

Subset selection

Reminder that the primal problem is the original constrained optimisation problem

$$\min_{\beta} \|Y - X\beta\|_2^2 \quad \text{s.t.} \quad \text{only } k \text{ coefficients are non-zero}$$

So instead of shrinking coefficients like Ridge, subset selection says to only use k features at most, set everything else to exactly zero.

The dual problem is a mathematically equivalent reformulation that moves the constraint into the objective as a penalty. Eg moving a max on the objective to min on the limits.

Strong duality means these two formulations give the exact same solution — for every k in the primal, there's a corresponding λ in the dual that gives identical β .

Back to the slide, SSR decreases as k increases, more features always improves fit on training data.

Are penalised and constrained forms equivalent though? Will see.

Subset selection seems like the Holy Grail because it has best L2 risk (MSE, called this because the average magnitude of the residuals use the L2 norm, which is Euclidean distance).

Has best prediction risk (expected error on unseen) and support recovery (finding the truly important features). But there's a major problem of computation.

Objective (SSR) is convex, constraint (only k non-zero) not convex.

Convex means like a bowl, any line drawn between two points on the curve sits above the curve.

$RSS = \|y - X\beta\|^2 = \sum (y_i - X_i'\beta)^2$ is a sum of squared terms which always creates a bowl shape. There is one global minimum and gradient descent is guaranteed to always find it and is computationally easy to solve.

But the constraint 'only k coefficients non-zero' is not convex, meaning there are many local optima and gradient descent can get stuck, so you'd need to check every possible combination of k features (which grows exponentially) to guarantee finding the best one.

Convexity = computationally tractable. Non-convexity = potentially exponentially hard to solve.

We will see that LASSO (which is convex) is preferred in practise.

$$\min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|_0$$

Given we've chosen a λ , how do we actually solve the Subset Selection problem.

There are three approaches:

Exhaustive search:

For a fixed model size s , check every possible combination of s features.

There are n choose s (nCs) combinations which grows explosively with p ; it is guaranteed to find the best solution but computationally infeasible for large p .

Greedy approach:

Forward selection — start with no features, keep adding the one that most reduces RSS, stop when penalty outweighs improvement

Backward selection — start with all features, keep removing the one that least hurts RSS

Bidirectional — combination of both

Faster but no guarantee to find the best solution.

Mixed integer programming:

Reformulates exhaustive search more cleverly.

Though overall better optimisation -> more danger of overfitting. The better we solve the Subset Selection problem exactly, the more risk of fitting noise.

Once we have chosen a method to solve for fixed λ , how do we choose λ itself?
The slide only lists some methods, so I will write them with one line explanation.

Cross-validation:

Split your data into chunks, train on some, test on others, pick the λ that performs best on the test chunks. Most practically used method — no strong assumptions needed.

Optimism correction:

Training error is always optimistically low because the model was fitted on that data. Optimism correction analytically estimates how much lower training error is vs test error and adjusts for it.

AIC and BIC:

Both follow the same structure:

fit quality penalty + complexity penalty

AIC penalises each extra parameter by 2

BIC penalises each extra parameter by $\log(n)$

Since $\log(n) > 2$ for $n > 8$ (basically always), BIC penalises complexity more harshly $\rightarrow$ tends to select simpler models than AIC.

Mallow's C_p :

Similar idea but scales the penalty by $\hat{\sigma}^2$ — the estimated noise variance. So if the data is very noisy, the penalty is harsher.

These are all trying to do the same thing — estimate test error without actually having test data. Cross-validation does it empirically, the others do it analytically.

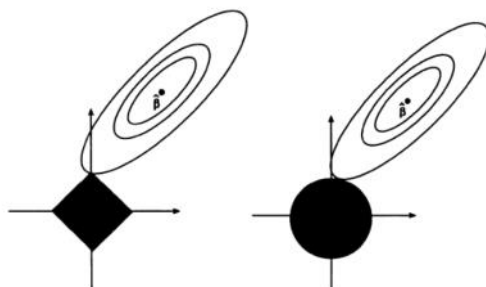


Figure 2 of "Regression Shrinkage and Selection via the Lasso" by Tibshirani, Journal of the Royal Statistical Society. Series B (Methodological), Vol. 58, No. 1 (1996)

The diamond shape is the geometry of the l_1 constraint $\|\beta\|_1 \leq c$. This is the convex relaxation of the non-convex Subset Selection constraint — replacing $\|\beta\|_0$ with $\|\beta\|_1$.

Diamond (LASSO) — the ellipse is very likely to touch at a corner, where one coefficient is exactly zero $\rightarrow$ sparsity

Circle (Ridge) — the ellipse touches on the smooth boundary, coefficients are shrunk but rarely exactly zero

This is the geometric reason why LASSO produces sparse solutions, exact zeros in coefficients. Ridge does not.

--

Three points about the diamond (l_1) shape:

RSS contours are likely to intersect the diamond at a corner

Corners correspond to exact zeros in some coefficients $\rightarrow$ sparsity

l_1 as opposed to l_0 (subset selection, sparsity) and l_2 (Ridge, computationally easy), is convex (easy to solve) and has corners which leads to sparsity, the others don't have one or the other.

LASSO (Least Absolute Shrinkage and Selection operator) is a regularised expression that solves:
 $\min_{\beta} (1/n) ||y - X\beta||^2 + \lambda ||\beta||_1$ like standard.

It's OLS but with L1 penalty added. This does:

Shrinkage — like Ridge, it pulls coefficients towards zero to prevent overfitting

Selection — unlike Ridge, it sets some coefficients to exactly zero, effectively removing those features from the model entirely

So LASSO is both a regression method and a feature selection method at the same time.

Ridge → shrinks all coefficients but keeps them all

Subset Selection → selects features exactly but computationally hard

LASSO → sits in between, shrinks coefficients AND performs feature selection, whilst remaining computationally tractable

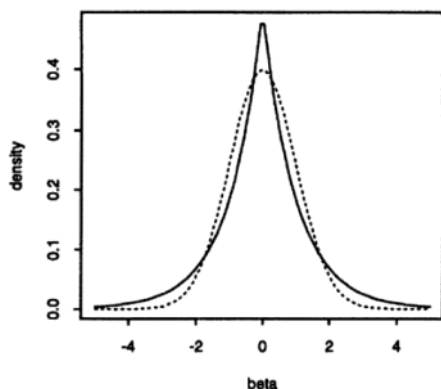
λ again controls the tradeoff — larger λ means more coefficients get set to zero.

Giving LASSO a Bayesian interpretation:

If we have a double-exponential (Laplace) prior on each β_j :

$$f(\beta_j) = \frac{1}{2\tau} \exp\left(-\frac{|\beta_j|}{\tau}\right)$$

Double exponential is where we have $\exp()$ and the modulus around the β_j , this looks like a tent, peaked at zero then falling off steeply on both sides, the solid line below:



Where the dotted is Ridge, L2.

Back to above.

Then the Bayes posterior mode solves:

$$\min_{\beta} (1/n) ||y - X\beta||^2 + (1/\tau) ||\beta||_1$$

Which is exactly LASSO with $\lambda = 1/\tau$.

Ridge → normal prior $N(0, \tau^2) \rightarrow$ l2 penalty $||\beta||^2$

LASSO → double-exponential prior → l1 penalty $||\beta||_1$

Standardisation and practical information about preprocessing your data before running LASSO.

Standardise x and y such that for y:

$$(1/n) \sum y_i = 0 \text{ -- i.e. demean y}$$

For each feature x_j :

$(1/n) \sum X_{ij} = 0 \rightarrow$ zero mean and $(1/n) \sum X_{ij}^2 = 1 \rightarrow$ unit variance

Demeaning y and x amounts to not penalising the intercept (demeaning makes the intercept 0).

Intercept β_0 shouldn't be penalised because it just captures the baseline level of y , not model complexity.

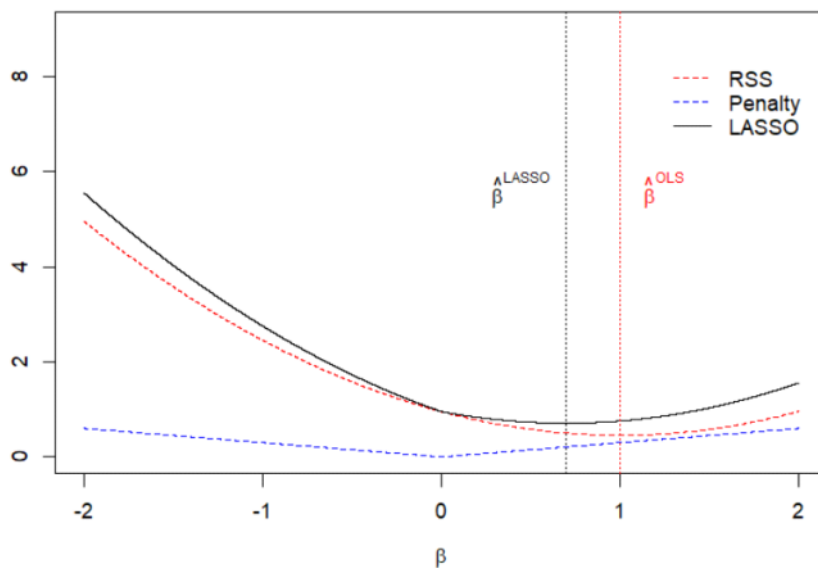
Unit variance for x makes all features comparable in scale.

For example income measured in £ are large and age are small numbers.

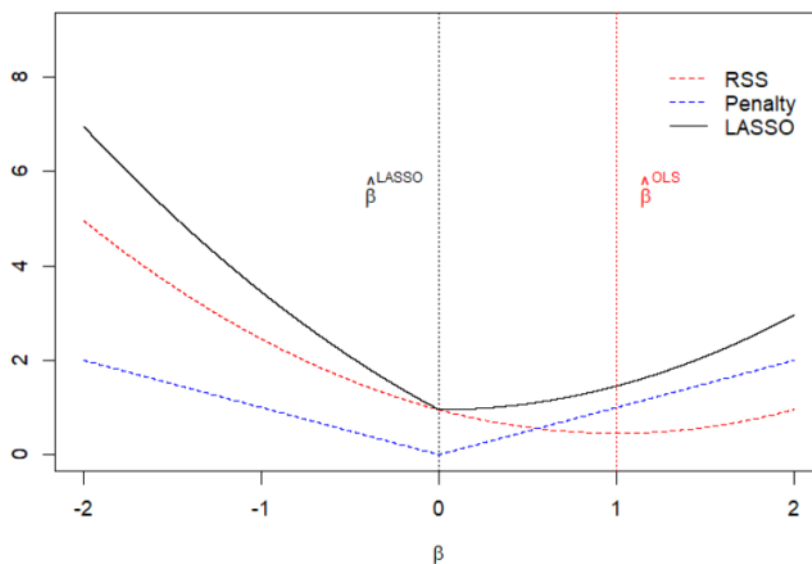
Without standardisation, the penalty $||\beta||_1$ would unfairly penalise the income coefficient more just because of its scale, not its true importance.

Compare the two following:

Objectives for $\lambda = 0.3$



Objectives for $\lambda = 1$

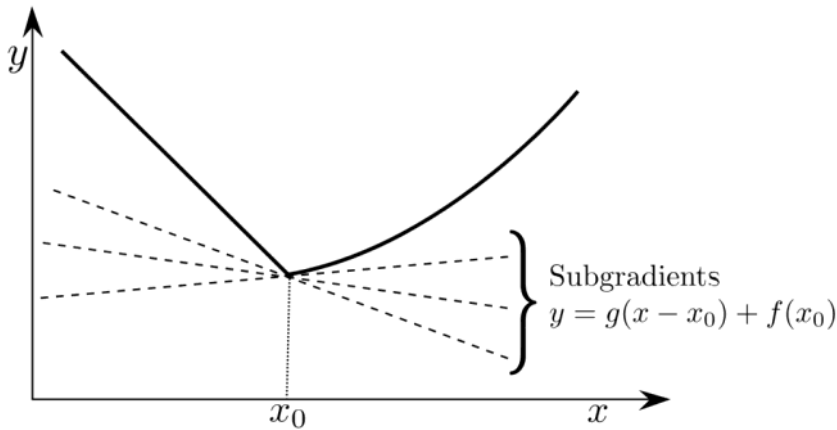


The LASSO solution $\hat{\beta}^L$ is where the black curve is minimised.

As λ increases, penalty goes from mild to strong, to the point where it is strong enough such that $\hat{\beta}^L$ is at zero. This is sparsity, going to zero. Meaning that feature has been removed from the model.

Pulls OLS estimator out towards zero.

Subgradients:



To minimise the LASSO objective we'd normally take the derivative and set it to zero. But $|\beta|$ has a kink at zero — it's not smooth there, so the regular derivative doesn't exist at that point. There's no single tangent.

$$y = g(x - x_0) + f(x_0)$$

Meaning any line with slope g is a subgradient at x_0 if it lies entirely below the function.

Instead of requiring a single derivative to be zero (as in OLS), LASSO requires that zero is contained within the set of subgradients at the solution.

Solving Univariate (single feature) LASSO:

Let the objective be:

$$f(\beta) = \frac{1}{2} \frac{1}{n} \sum_{i=1}^n (y_i - x_i \beta)^2 + \lambda |\beta|$$

Subgradient of $|\beta|$:

$$\frac{\partial}{\partial \beta} |\beta| = \begin{cases} 1 & \text{if } \beta > 0 \\ [-1, 1] & \text{if } \beta = 0 \\ -1 & \text{if } \beta < 0 \end{cases}$$

Subgradient of full objective:

$$\frac{\partial}{\partial \beta} f(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - x_i \beta)(-x_i) + \lambda \frac{\partial}{\partial \beta} |\beta|$$

Set the RHS of the above to zero and simplify, using $(1/n) \sum x_i^2 = 1$ from standardisation:

$$0 = -\hat{\beta}^{OLS} + \beta + \lambda \frac{\partial}{\partial \beta} |\beta|$$

Soft-thresholding solution:

$$\hat{\beta}^{\text{LASSO}} = S(\hat{\beta}^{\text{OLS}}, \lambda) := \begin{cases} \hat{\beta}^{\text{OLS}} - \lambda & \text{if } \hat{\beta}^{\text{OLS}} \geq \lambda, \\ 0 & \text{if } \hat{\beta}^{\text{OLS}} \in [-\lambda, \lambda], \\ \hat{\beta}^{\text{OLS}} + \lambda & \text{if } \hat{\beta}^{\text{OLS}} \leq -\lambda. \end{cases}$$

For $\hat{\beta}^{\text{L}}$ to be optimal, zero must be in the subgradient of $f(\beta)$.

For instance in case 1:

$\hat{\beta}^{\text{L}} > 0$; The subgradient of $|\beta| = 1$, so: $\beta^{\text{L}} = \beta^{\text{OLS}} - \lambda$

This only makes sense if $\beta^{\text{OLS}} > \lambda$ (otherwise β^{L} would be negative, contradicting our assumption)

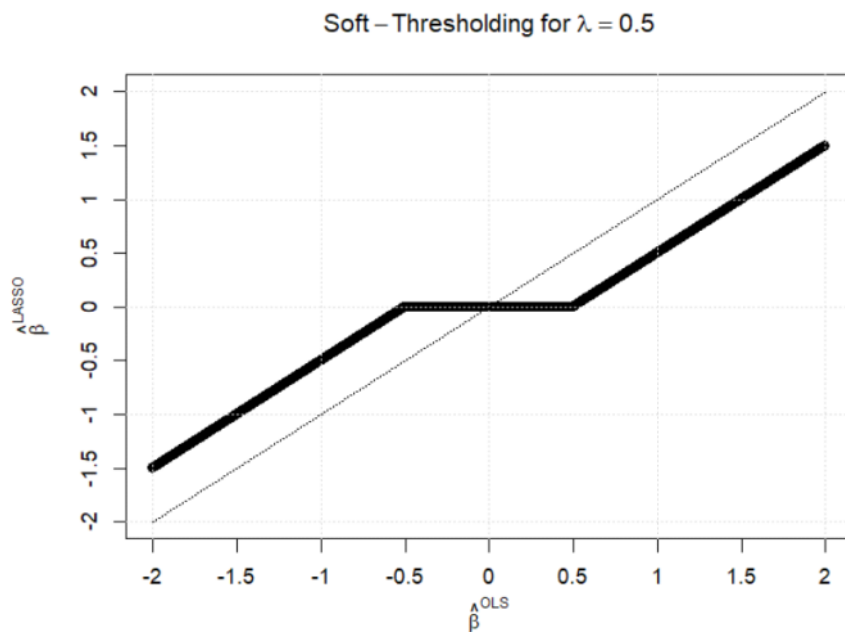
Essentially if $\hat{\beta}^{\text{L}}$ is within λ : $|\beta^{\text{OLS}}| \leq \lambda$

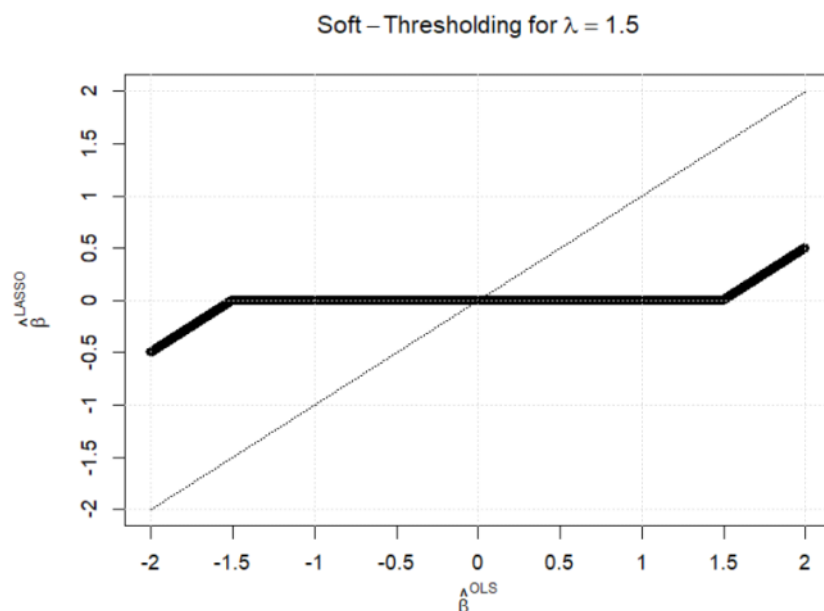
It is so small that it will just get reduced to zero.

If $\hat{\beta}^{\text{L}}$ is positive and greater than lambda, then it passes the threshold so that it actually matters, subtract λ so that it shrinks but still exists.

If $\hat{\beta}^{\text{L}}$ is negative and but more negative than negative lambda, then it passes the threshold so that it actually matters, add λ so that it shrinks towards zero but still exists.

Visually:





The flat region at zero gets wider as λ increases, more OLS coefficients get killed.
 This is LASSO performing automatic feature selection — as λ increases it progressively eliminates features until eventually all coefficients are zero (like we saw at $\lambda = 1$ before).

Now with multiple features:

Instead of optimising over all β simultaneously, we cycle through one coefficient at a time, fixing all others and solving the univariate LASSO problem (above).

Initialise $\beta = 0$ and residuals $r = y - \bar{y}$, standardise X .

Outer loop cycle over all features $j=1, \dots, p$

Add back $x_j \beta_j$ to residuals r

Regress r on x_j to get β_j^{OLS}

Apply soft-thresholding: $\beta_j = S(\beta_j^{\text{OLS}}, \lambda)$ to get updated LASSO estimate.

Remove $x_j \beta_j$ from residuals r (put β_j 's new contribution back into the residuals ready for the next feature.)

Optional inner loop — repeat but only over features where $\beta_j \neq 0$ (speeds things up)

Set intercept $\beta_0 = \bar{y}$ (not penalised)

Rescale coefficients back

Note on the 'add back' part:

At any point, $r = y -$ (contribution of all other features)

To optimise β_j , we need to know what r looks like as if β_j wasn't in the model yet.

But r already has β_j 's contribution baked into it from the previous iteration.

So adding it back: $r \leftarrow r + x_j \beta_j$ removes β_j 's current contribution from the residuals. Now regress r on x_j and estimate what β_j should be, given everything else.

Add-back just means temporarily undo β_j 's effect so we can re-estimate it cleanly.

The 'rescale coefficients back' part means:

Before running LASSO we standardised X such that each feature had: $(1/n) \sum X_{ij}^2 = 1$
 making all features comparable for penalisation.

So now our coefficients are on the standardised scale, not the original scale. So rescale:

● rescale: $\beta_j = \beta_j / \sqrt{n} \|x_j\|_2$

Implementation: $\lambda_{\max}$

We will see that there is a maximum value of λ , where beyond which all coefficients are zero.

If:

$$\lambda \geq \lambda^{\max} = \max_{j=1,\dots,p} \left| \hat{\beta}_j^{\text{univariate OLS}} \right|,$$

Then $\beta^L = 0$ (all coefficients zero)

If $\lambda < \lambda_{\max}$

Then $\beta^L \neq 0$ (at least some coefficients non-zero)

From before with univariate LASSO, a coefficient survives LASSO only if $|\hat{\beta}_j^{\text{OLS}}| > \lambda$. So if λ is larger than every single OLS coefficient, none of them can clear the hurdle $\rightarrow$ all set to zero.

$\lambda_{\max}$ is simply the largest OLS coefficient in absolute value — the point at which even the strongest feature gets killed.

Final slide.

In practice we don't just solve LASSO for one λ — we solve it for a whole sequence of λ s to find the best one.

The algorithm, (glmnet approach):

Compute $\lambda_{\max}$ (the largest OLS coefficient in absolute value)

Set $\lambda_{\min} = \epsilon \cdot \lambda_{\max}$ where $\epsilon = 0.0001$ (very small)

Take an equidistant sequence of length l between $\log(\lambda_{\min})$ and $\log(\lambda_{\max})$

Use the exponential of this sequence as the λ sequence

Loop over λ s starting from the largest:

At λ_1 (largest) $\rightarrow$ start from $\beta = 0$

At each subsequent $\lambda_i \rightarrow$ start from previous solution $\beta^L(\lambda_{i-1})$

Explaining:

First, use log scale because at very small λ the solution changes slowly, at larger λ it changes quickly. The log scale spaces the λ s more evenly in terms of their effect on the solution.

At $\lambda_{\max}$, solution is $\beta = 0$ (trivial starting point)

As λ decreases, solution changes gradually

Using the previous solution as a warm start makes each step much faster to compute.

We want to find the best λ for LASSO. Rather than guessing, we solve LASSO for many values of λ and then pick the best one later.

We are searching between the max λ and a very small value.

Rather than trying every possible λ , we pick say 100 evenly spaced values on a log scale between $\lambda_{\min}$ and $\lambda_{\max}$.

For each λ , solve LASSO using coordinate descent, using the previous ($\lambda_2 \rightarrow$ start from solution at λ_1 , so on).

Each time λ decreases slightly, the solution only changes a little. So starting from the previous solution means coordinate descent converges very quickly rather than starting from scratch each time.

This gives you a full path of solutions from $\beta = 0$ (at $\lambda_{\max}$) all the way to near-OLS (at $\lambda_{\min}$).

One $\hat{\beta}^{\text{LASSO}}$ for each λ value. You then use cross-validation (the list before, eg AIC/BIC) to pick which λ gave the best model.

The above steps algorithm 'is' what LASSO is.

Reminder that we need it when: standard OLS breaks down when $p > n$ (more features than observations) because $X'X$ becomes non-invertible. LASSO solves this by penalising complexity.

Optimisation

09 June 2026 00:49

The goal of supervised ML: low test error, achieved in two steps:

Small test error, get the model to fit the training data well, **where optimisation is often necessary**.

Small gap between training and test error (generalisation gap), ensure the model generalises well to the new data.

So this second step is another concern and not regarding optimisation.

In fact, these two steps can conflict -- the better you optimise on training data, you risk overfitting.

- d is the dimension
- we are given functions $f_0, f_1, \dots, f_m : \mathbb{R}^d \rightarrow \mathbb{R}$
- f_0 is the objective function
- $(f_j)_{j=1, \dots, m}$ are the constraint functions

$$\inf_{x \in \mathbb{R}^d} f_0(x) \text{ subject to } f_j(x) \leq 0 \text{ for } j = 1, \dots, m$$

d is how many variables we are optimising over

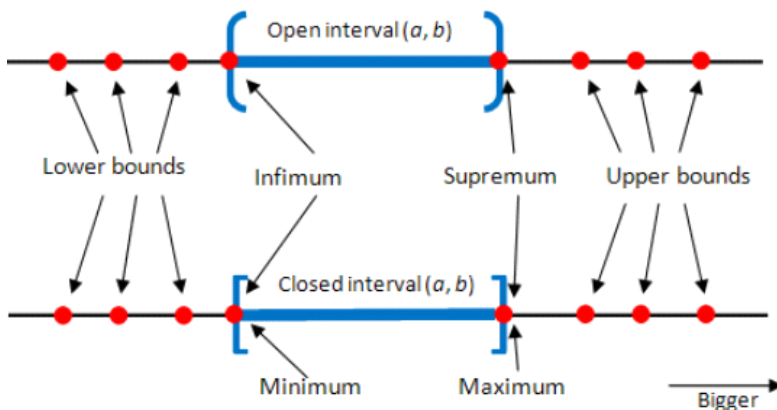
The collection of functions map a d -dimensional input and output a single number

f_0 is the objective function, what we are trying to minimise

Constraint functions define boundaries our solution must stay within

Then at the bottom is the general optimisation problem. Find x that makes $f_0(x)$ as small as possible, while satisfying all constraints $f_j(x) \leq 0$.

$\inf$ means the greatest lower bound, which handles cases where the minimum might not actually be reached.



Infimum: The greatest lower bound of a set. It doesn't necessarily have to belong to the set itself (though it always does for bounded, closed sets).

Minimum: The smallest actual member of the set. If a set has a minimum, it is equal to the infimum.

Open Sets (e.g., $(3, 5)$): The infimum is still 3. However, because the set is open (excludes its boundary), the number 3 is never actually reached. The infimum exists, but the minimum does not exist.

More notation:

x is feasible if it satisfies all the constraints: $f_j(x) \leq 0$ for $j = 1, \dots, m$

The feasible set D is the collection of all feasible points: $D = \{x \mid f_j(x) \leq 0 \text{ for } j = 1, \dots, m\}$. Solution must live here.

The optimal value f^* is the smallest possible value of f_0 within D : $f^* := \inf\{f_0(x) \mid x \in D\}$

A point x^* is optimal if:
 It's feasible ($x^* \in D$)
 It achieves the optimal value ($f_o(x^*) = f^*$)

X_{opt} is the set of all optimal solutions — there could be more than one.

$\tilde{x}$ is locally optimal if it's the best solution within a small neighbourhood of size ϵ around it — but there might be better solutions further away.

An algorithm might find a locally optimal solution that isn't globally optimal, this is why convexity is so desirable, otherwise it is a challenge.

A function $d: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a distance measure if it satisfies four properties for all x, y :

1. $d(x, y) \geq 0$

Distance is always non-negative — you can't have negative distance between two points.

2. $d(x, y) = 0$ if and only if $x = y$

Distance is zero only when the two points are the same point.

3. $d(x, y) = d(y, x)$

Distance is symmetric — the distance from x to y is the same as from y to x .

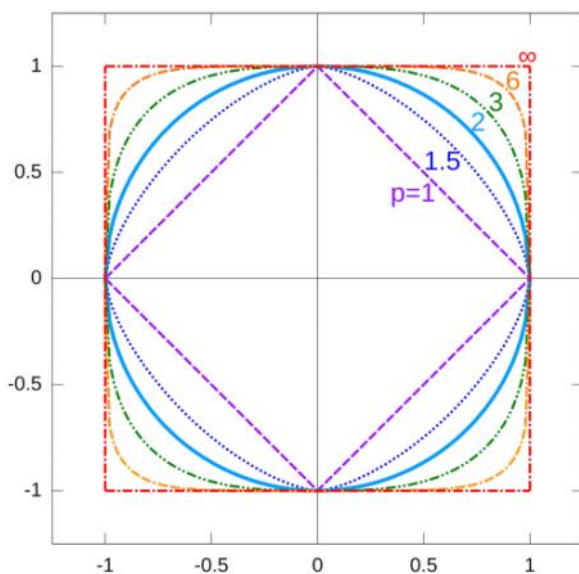
4. $d(x, y) \leq d(x, z) + d(y, z)$

The triangle inequality — the direct distance between two points is never longer than going via a third point.

The above is simple just formalising.

We needed a notion of 'nearby' closeness to define a neighbourhood of size ϵ around $\tilde{x}$. The distance measure formalises this.

Different choices of distance measure give different notions of "closeness", which we will find out.



L_p unit balls, where each unit ball is simply the set of all points within distance 1 from the origin, under a given metric.

What L_p is generally since we have seen it a few times now:

The L_p norm of a vector β is defined as:

$\|\beta\|_p = (|\beta_1|^p + |\beta_2|^p + \dots + |\beta_p|^p)^{1/p}$. It's just a way of measuring size or length of a vector, but with a different emphasis on the components depending on p .

Eg if $\beta = (3, 4)$:

l_1 ($p=1$):

$$||\beta||_1 = |3| + |4| = 7$$

Just adds up absolute values — treats all components equally

l_2 ($p=2$):

$$||\beta||_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

Standard Euclidean distance — "as the crow flies"

l_∞ ($p=\infty$):

$$||\beta||_\infty = \max(|3|, |4|) = 4$$

Just takes the largest component

As p increases, the norm becomes increasingly dominated by the largest component and pays less attention to smaller ones.

As p decreases towards 1, all components get more equal treatment. This is what is meant by emphasis on different components depending on p .

Hence, l_1 (LASSO) promotes sparsity by treating all components equally, it's "willing" to set small coefficients to exactly zero rather than letting large ones dominate. Ridge (l_2) is more influenced by large coefficients and so shrinks everything but rarely zeros anything out.

The unit ball is all points where $||\beta||_p = 1$. The shape of p determines the shape of the feasible region.

Setup & Solution Methods for optimisation

Methods are classified by the degree of derivatives they use. Higher order of the method you are using means you get more information about the function and faster convergence, but at a cost.

0th order methods, this method requires no derivatives, only needs to evaluate $f_0(x)$ itself at different points.

-Grid Search — try many values of x , pick the best one

-Simulated Annealing — a smarter random search

1st order methods — uses the first derivative (gradient):

-Gradient Descent — follow the steepest downhill direction

Needs $\nabla f_0(x)$ — tells you which direction to step.

2nd order methods — uses the second derivative (Hessian):

-Newton-Raphson — uses curvature information to take smarter steps

Needs $H(f_0(x))$ — tells you not just direction but how curved the function is.

Higher order methods converge faster but not every function can be differentiated and the curse of dimensionality is that in high dimensions, computing gradients, especially Hessians, become very expensive.

From Claude: In ML we typically use 1st order methods (gradient descent) because computing the Hessian in high dimensions is prohibitively expensive, but pure function evaluation (0th order) is too slow. Gradient descent hits the sweet spot.

Suppose $x \in U = [0, 1]^d$

Meaning x lives in the unit hypercube $[0, 1]^d$ (a d -dimensional box where every value is between 0 and 1).

The algorithm:

Draw many values of x across this space, either on a regular grid or randomly
Evaluate $f_0(x)$ at each point
Pick the x that gives the lowest value of f_0

The Lipschitz condition states if f is continuous: $|f(x) - f(y)| < L|x - y|$

Then the function can't change too wildly — if two points are close together, their function values are also close. L measures how "fast" the function can change. I.e. Lipschitz-continuity says the change in function value is always proportional to the change in input, bounded by a constant L . L is like the speed limit on how fast the function can change. If you know f is Lipschitz with constant L , and your grid points are spaced ϵ apart, then between any two grid points the function can't change by more than $L\epsilon$.

Under this condition, the error of grid search is bounded by: $|f^* - f(x^*)| \leq L\epsilon$ i.e.e the true optimum gives an answer within $L\epsilon$ of the true optimum

So by making the grid fine enough (small ϵ), you can get arbitrarily close to the true optimum.

Grid search is simple but has a massive problem in high dimensions — if you have d dimensions and want 10 grid points per dimension, you need 10^d evaluations total. For $d=100$ this is completely infeasible. Curse of dimensionality.

Simulated Annealing

Rather than evaluating a fixed grid, simulated annealing does a guided random search — it explores the space intelligently, occasionally accepting worse solutions to avoid getting stuck.

Step 1 — Initialise:

Start at some arbitrary point x_0

Choose a sequence of "temperatures" (T_i)

Choose a sequence of "neighbourhood sizes" (s_i) $\rightarrow 0$

Choose a density $f_T()$

Step 2 — Propose a move:

Draw a random step Z from the density $f_{T_i}()$

If Z is too small ($d(0, Z) < s_i$), redraw

Define candidate new point $Y = X + Z$

Step 3 — Accept or reject:

$P[X^{\text{next}} = Y | X, T] = \min(1, (f_0(X) - f_0(Y))/T)$

If Y is better than $X \rightarrow$ always accept

If Y is worse $\rightarrow$ sometimes accept, with probability depending on temperature T

The acceptance probability says if Y is better than X -- ($f_0(Y) < f_0(X)$), then $f_0(X) - f_0(Y) > 0$, probability 1, always accept.

If worse, $f_0(X) - f_0(Y) < 0$, probability is small but positive, sometimes accept.

A high T value (more below) divides by a larger number, probability closer to 1, more likely to accept bad moves in the pursuit of the optimal point. Vice versa.

Step 4 — Update T and s or terminate

The occasionally accepting worse solutions stops the algorithm from getting trapped in a local minimum.

--

Going to now explain the above more informally.

Going down a hill trying to find the lowest point, grid search just checks most points that exist.

Simulated annealing instead walks around intelligently.

First, a density function $f_T()$ describes the probability distribution from which you draw your proposed step Z .
High $T \rightarrow$ wider/flatter distribution $\rightarrow$ large steps more likely
Low $T \rightarrow$ narrower distribution $\rightarrow$ mostly small steps
Common choices are a normal distribution, or uniform over neighbourhood, all step sizes equally likely.

The T (temperature) parameter controls the spread of this distribution:

High $T \rightarrow$ density is wide $\rightarrow$ big jumps likely

Low $T \rightarrow$ density is narrow $\rightarrow$ only small steps

The temperature starts high and gradually decreases, the cooling schedule. There are different choices to how this happens, be linear, geometric, logarithmic in the way the temp cools. If you cool too fast you might get stuck.
Early on, large neighbourhood, big jumps and wide exploration.
Later on, opposite. Tiny steps to fine-tune.

Overall, simulated annealing is better when the space is too large for grid search, dimension d .

If the function is non-convex with multiple local minima, if you have some structure to exploit (simulated annealing searches in promising areas rather than wasting evaluations in obviously bad areas).

Grid search is better if the space is small and low dimensional, you need guaranteed coverage, or there is no good starting points that exists. Simulated annealing starting in a bad place can take a while to get somewhere useful.

1st order methods, gradient descent.

At a local optimum $\tilde{x}$, the gradient is zero: $\nabla_x f_0(\tilde{x}) = 0$

The algorithm:

Start at some initial point x_0

Choose a sequence of step sizes (α_i)

Repeatedly update:

$$x_{i+1} = x_i - \alpha_i \nabla_x f_0(x_i)$$

Step in the opposite direction of the gradient to go downhill, step size α_i controls how far you move.

Taking small enough steps guarantees you always reduce the loss — you won't overshoot the minimum.

Linear convergence on well-behaved problems -- on nice smooth problems, the error reduces by a constant factor each step — predictable and reliable.

Gradient descent converges to a point where $\nabla f_0 = 0$ — but this could be a local minimum, **not necessarily global**.

How to choose step size/learning rate:

Fundamentally:

Too large $\alpha_i \rightarrow$ might overshoot the minimum $\rightarrow$ objective increases

Too small $\alpha_i \rightarrow$ guaranteed to decrease but very slow to converge

Four step size rules:

Can keep it constant, $\alpha_i = \alpha = 0.01$. Easy to implement but requires careful tuning to not cross the two issues above.

Diminishing step size: $\sum \alpha_i = \infty$ and $\sum \alpha_i^2 < \infty$

$\sum \alpha_i = \infty \rightarrow$ steps are large enough to reach the minimum eventually

$\sum \alpha_i^2 < \infty \rightarrow$ steps shrink fast enough to settle at the minimum rather than bouncing around it

Exact line search:

$$\alpha_i = \operatorname{argmin}_{\alpha} f(x_i + \alpha \nabla_x f(x_i))$$

At each step, find the perfect step size by minimising the objective along the gradient direction. Optimal but computationally expensive.

Argmin for a function means:

min → gives you the value of the function at the minimum

argmin → gives you the input that achieves the minimum

So the above says to find the value of the step size α that minimises the objective function when stepping from x_i in the gradient direction"

This is usually never used, too expensive for millions of parameters.

Backtracking line search:

Start with a large step size

Keep reducing it until the objective actually decreases

Armijo-Goldstein is a specific rule for when to stop reducing. Is a compromise between exact line search and constant step size.

2nd order, Newton-Raphson

Set up is the same as gradient descent but the update rule is different:

At a local optimum $\tilde{x}$, the gradient is zero:

$$\nabla_x f_0(\tilde{x}) = 0$$

Start at x_0 .

$$\bullet \quad x_{i+1} = x_i - (H_{f_0}(x_i))^{-1} \nabla_x f_0(x_i).$$

Compared to gradient descent: $x_{i+1} = x_i - \alpha_i \cdot \nabla_x f_0(x_i)$

The difference is that instead of a scalar step size α_i , Newton-Raphson uses $(H_{f_0}(x_i))^{-1}$ — the inverse of the Hessian matrix.

The Hessian H is a matrix of second derivatives — it tells you how the gradient is changing in every direction. It captures the curvature of the function.

The Hessian tells you how curved the function is at your current point:

High curvature → take a small step (steep bowl, minimum is nearby)

Low curvature → take a large step (flat surface, minimum is far away)

No need to manually choose α_i .

Newton-Raphson converges much faster than gradient descent — often in far fewer steps. But computing and inverting the Hessian is extremely expensive in high dimensions (remember $p > n$ problems from the regularisation deck!). For a model with p parameters, the Hessian is a $p \times p$ matrix — inverting this when p is large is practically infeasible.

This is why gradient descent dominates in ML despite being slower.

That was unconstrained optimisation, now constrained optimisation, meaning with constraints and cannot just optimise the objective function freely.

A **set** S is convex if for any two points x and y in S , the straight line between them is entirely within S :

$$\lambda x + (1-\lambda)y \in S \text{ for all } \lambda \in [0,1]$$

Has no dents or holes.

A **function** f is convex if for any two points x and y :

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$$

The function value at any point between x and y is below or on the straight line connecting $f(x)$ and $f(y)$.

An optimisation **problem** is convex if all the functions involved are convex — both the objective f_0 and all constraints $f_1, \dots, f_m$.

$$\inf_{x \in \mathbb{R}^d} f_0(x) \text{ subject to } f_j(x) \leq 0 \text{ for } j = 1, \dots, m$$

is convex if $f_0, f_1, \dots, f_m$ are all convex.

Convex optimisation key properties:

A convex function is differentiable "almost everywhere":

Except possibly at kinks (like $|\beta|$ at zero — exactly what we saw with LASSO subgradients). This means gradient based methods work almost everywhere.

The feasible set of a convex problem is convex:

Any local minimum is a global minimum.

This is the most important property — in a convex problem you can never get stuck in a bad local minimum.

Primal and Dual problem

The primal problem is the original constrained problem:

$$\inf_{x \in \mathbb{R}^d} f_0(x) \text{ subject to } f_i(x) \leq 0 \text{ for } i = 1, \dots, m$$

The dual problem moves the constraints into the objective using multipliers λ :

$$\sup_{\lambda \in \mathbb{R}^m} \inf_{x \in \mathbb{R}^d} f_0(x) + \sum_{i=1}^m \lambda_i f_i(x).$$

Each λ_j (Lagrange multipliers) is attached to one constraint $f_j(x) \leq 0$. Represents the cost of violating that constraint.

$\lambda_j = 0 \rightarrow$ constraint j is completely ignored

$\lambda_j = \text{small} \rightarrow$ slight penalty for violating constraint j

$\lambda_j = \text{large} \rightarrow$ heavy penalty for violating constraint j

λ too small $\rightarrow$ constraint barely enforced $\rightarrow$ solution might violate it $\rightarrow$ loose lower bound

λ too large $\rightarrow$ constraint over-enforced $\rightarrow$ solution too conservative $\rightarrow$ loose lower bound

λ just right $\rightarrow$ constraint enforced perfectly $\rightarrow$ tightest lower bound $\rightarrow$ equals f^*

We're not picking λ — we're optimising over it to find the best value. The dual problem finds λ^* automatically as part of the solution. Represents the theoretically optimal price for the constraint

But in Ridge/LASSO — λ IS chosen by the user.

$$g(\lambda) = \inf_{x \in \mathbb{R}^d} \{f_0(x) + \sum \lambda_i f_i(x)\} \text{ (the part within the sup is function } g)$$

Properties of the dual:

$$f^* \geq g(\lambda) := \inf_{x \in \mathbb{R}_+^d} f_0(x) + \sum_{i=1}^m \lambda_i f_i(x).$$

Properties of the Dual:

First:

$g(\lambda)$ is always concave as a function of λ — meaning it has an upside down bowl shape.

Concave functions have a single peak $\rightarrow$ easy to maximise

Even if the original primal problem is horrible and non-convex, the dual is always nicely concave

This means the outer sup is always tractable

In the dual problem:

inf (infimum) → the greatest lower bound, essentially the minimum

sup (supremum) → the least upper bound, essentially the maximum

They are used instead of min/max for technical reasons when the min/max might not actually be reached.

The inner inf says for these specific values of λ , find the x that minimises the objective plus the weighted constraints. This gives a number $g(\lambda)$ — different for each choice of λ .

The outer sup, over all λ finds the one that gives the largest $g(\lambda)$.

The inner inf is trying to solve the original problem, trying to minimise the objective, eg training error.

For a given set of λ s, we find the best possible x .

Solving from below, the dual always gives a lower bound, so we want the largest value of our lower bound, to be nearest f^* , the optimal. Meaning our answers here are telling us that the f^* is this value or higher.

Second:

$$f^* \geq g(\lambda) := \inf_{x \in \mathbb{R}_+^d} f_0(x) + \sum_{i=1}^m \lambda_i f_i(x).$$

Which just says $f^* \geq g(\lambda)$ for any λ (and then writes out the function on the right, but the LHS is the main part).

No matter what λ you choose, $g(\lambda)$ will always be below or equal to the true optimal value f^* .

The dual always gives a lower bound on the true optimal value f^* . So $g(\lambda)$ is always sitting below (or equal to) the true answer.

This is because for any feasible x (satisfying constraints) $f_i(x) \leq 0$. So adding $\lambda_i f_i(x)$ with $\lambda \geq 0$ can only decrease the objective — meaning the dual is always lower than the true answer.

Third:

The duality gap: $0 \leq f^* - g^*$

The difference between the primal optimal f^* and dual optimal g^* is called the duality gap. It's always ≥ 0 because of property 2, basically just repeating property two in order to define strong duality.

Strong duality means this gap = 0:

$$f^* - g^* = 0 \rightarrow f^* = g^*$$

Meaning primal and dual give exactly the same answer.

If strong duality holds, solve whichever of primal or dual is easier — you'll get the same answer either way.

If two conditions hold:

The primal problem is convex

Slater's condition holds

Then strong duality holds — meaning $f^* = g^*$ (primal and dual give identical answers).

Slater's condition:

There exists at least one point $x \in \mathbb{R}^d$ such that for all constraints:

$$f_i(x) < 0 \text{ (strictly less than zero)}$$

"There exists at least one point that strictly satisfies all constraints — not just barely, but with room to spare."

It rules out degenerate cases where:

The feasible set is empty (no valid solutions exist)

The feasible set has no interior (constraints are so tight there's only one possible point)

--

In the primal problem — hard constraints: $\inf_{\{x\}} f_0(x)$ subject to $f_j(x) \leq 0$

Here satisfying a constraint means strictly staying within the boundary: $f_j(x) \leq 0$

In the dual problem — soft penalties: $\inf_{\{x\}} f_0(x) + \sum \lambda_i f_i(x)$

There are no hard constraints anymore. Instead constraints are incorporated as penalties via λ .

Here you could technically choose any x — including ones that violate constraints — but violating them costs you through the penalty term.

So there's no "satisfying" in the hard sense — just a tradeoff between objective value and constraint violation cost.

The dual's soft penalties are perfectly calibrated (via λ^*) so that the optimal solution happens to satisfy all constraints exactly anyway — even without hard enforcement.

Primal → constraints are walls you cannot cross

Dual → constraints become costs that discourage crossing

KKT Conditions

KKT (Karush-Kuhn-Tucker) conditions are a set of necessary and/or sufficient conditions that must hold at the optimal solution (x^*, λ^*) of a constrained optimisation problem. Think of them as a checklist — if a solution is truly optimal, it must satisfy all four.

KKT conditions give us a way to verify whether a solution is optimal without having to search exhaustively.

1 Stationarity

2 Primal feasibility

3 Dual feasibility

4 Complementary slackness

1

$$0 \in \nabla_x f_0(x) + \sum_j \lambda_j \nabla_x f_j(x)$$

The gradient of the objective plus the weighted constraints equals zero at the optimal point.

This uses subgradients ($\in$ rather than $=$) to handle non-smooth functions like $|\beta|$ in LASSO.

2

Primal Feasibility: $x \in D$

The solution must be feasible — it must satisfy all the original constraints. You can't be optimal if you're outside the allowed region.

3

Condition 3 — Dual Feasibility: $\lambda \geq 0$

All Lagrange multipliers must be non-negative. This ensures the penalties are actually penalising constraint violations rather than rewarding them. This is because the constraints functions are negative themselves.

4

Complementary Slackness: $\lambda_j f_j(x) = 0$ for all $j = 1, \dots, m$

For each constraint j , either:

$\lambda_j = 0 \rightarrow$ the constraint is inactive (not binding) — the solution is comfortably inside the boundary

$f_j(x) = 0 \rightarrow$ the constraint is active (binding) — the solution is exactly on the boundary

So either: $\lambda_j = 0$, or $f_j(x) = 0$

They can't both be non-zero simultaneously.

If you are naturally inside the acceptable boundary ($f_j(x) < 0$), then there is no penalty so $\lambda_j = 0$, so this multiplication equals zero without both being zero. If you're within the boundary, no need for a penalty, as it has not been violated.

$f_j(x) \leq 0$ is the constraint — e.g. $\|\beta\|^2 - c \leq 0$ for Ridge
 $f_j(x) = 0$ means you're sitting exactly on the boundary
 $f_j(x) < 0$ means you're strictly inside the boundary

At the optimal solution, active constraints (binding) have non-zero λ — they're genuinely constraining the solution.

Inactive constraints (not binding) have $\lambda = 0$ — they're irrelevant at the optimum.

This is why in Ridge, $\lambda = 0$ gives OLS (constraint inactive) and $\lambda > 0$ shrinks coefficients (constraint active).

KKT theorem, when are the necessary and when sufficient.

Necessity: If strong duality holds AND x^* is optimal in the primal problem, then there exists a λ^* such that (x^*, λ^*) satisfy the KKT conditions.

If you've found the true optimal solution x^* , then the KKT conditions must hold at that point — you can't have a true optimum that violates them.

Sufficiency:

If (x^*, λ^*) satisfy the KKT conditions, then:

Strong duality holds

x^* is optimal in the primal

λ^* is optimal in the dual

If you find a point that satisfies all four KKT conditions, you're guaranteed it's the optimal solution — for both primal and dual simultaneously.

Every optimal solution satisfies KKT (necessity), every KKT solution is optimal (sufficient).

Instead of searching exhaustively for the minimum, you can just:

Write down the four KKT conditions

Solve them algebraically

The solution is guaranteed to be optimal

Is how we derived the LASSO soft-thresholding solution in the regularisation deck.

Application to supervised ML:

The objective is the training error, average loss over all training observations:

$$f_0(x) = \frac{1}{n} \sum_{i=1}^n l(Y_i, f(X_i)),$$

Where:

Y_i is the true outcome for observation i

$f(X_i)$ is the model's prediction for observation i

$l(Y_i, f(X_i))$ is the loss function — measuring how wrong the prediction is

$$f_0(x) \approx E[l(Y, f(X))]$$

Training error is just a sample approximation of the true expected loss, above expectation.

The derivative of a sum is the sum of a derivative.

Eg if you have: $g(x) = a_1(x) + a_2(x) + \dots + a_n(x)$; then $g'(x) = a_1'(x) + a_2'(x) + \dots + a_n'(x)$

f_0 is a linear average of individual losses:

$$\nabla_x f_0(x) = (1/n) \sum_i \nabla_x l(Y_i, f(X_i))$$

The gradient of the average = average of the gradients. This is important because it means:

Computing the gradient is just averaging individual gradients

This is computationally straightforward

And leads naturally to stochastic gradient descent on slide 27

Stochastic gradient descent

$\nabla_x l(Y_i, f(X_i))$ for every single observation can be expensive, especially if n is in the millions.

The training error is itself just an approximation of the true expected loss:

$$(1/n) \sum_i l(Y_i, f(X_i)) \approx \mathbb{E}[l(Y, f(X))]$$

So if the full average is already just an approximation, why not use a smaller random sample to approximate it even more cheaply?

Use a random subset $B \subset \{1, \dots, n\}$

$$\frac{1}{|B|} \sum_{b=1}^{|B|} l(Y_{B_b}, f(X_{B_b})) \approx \mathbb{E}[l(Y, f(X))]$$

Average over a random batch of B size $|B|$ (batch size)

Higher $|B|$ leads to a more accurate estimate of error

Lower is cheaper to compute though

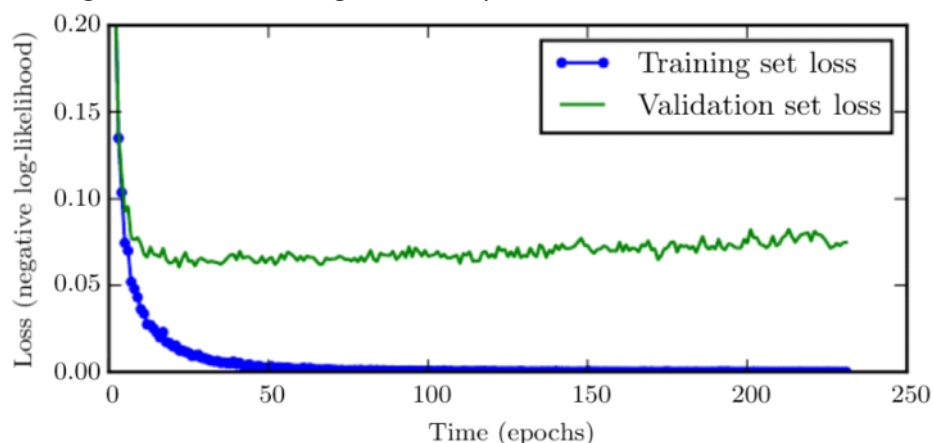
In practise; choose $|B|$ depending on hardware

Shuffle the data once and then just work through it in blocks

An epoch is one complete pass through the entire training dataset. Rather than adjusting the learning rate after every mini-batch, wait until you've seen all the data once.

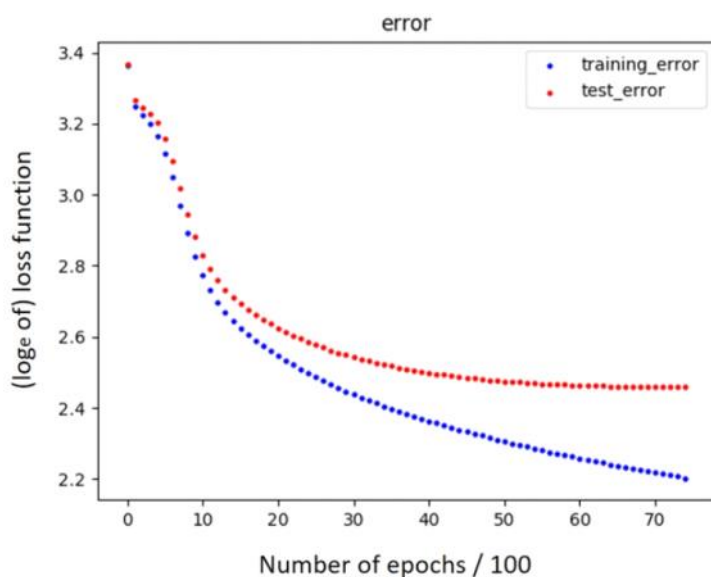
SGD is what makes training large ML models (eg NNs) computationally feasible. We accept a bit of noise in exchange for huge computational saving.

Training and test loss during iterative optimisation



Demonstrating overfitting.

The model keeps improving on training data but stops improving and even worsens on unseen.



This is from a (different) paper.

Making the case for early stopping:

Optimisation is costly and leads to overfitting

Early stopping saves computational time and prevents overfitting simultaneously. Seems to be no trade-off here.

These slides motivate why we'd deliberately stop optimisation early rather than running it to convergence, also links to the regularisation slides, probably should've done this first lol.

--

Early Stopping Algorithm, Alg. 7.1 in "Deep Learning"

Input: Initial parameters θ_o , patience p , steps between evaluations n

Output: Best parameters θ^* , best number of training steps i^*

$\theta \leftarrow \theta_o, i \leftarrow 0, j \leftarrow 0, v \leftarrow \infty, \theta^* \leftarrow \theta, i^* \leftarrow i$

while $j < p$ **do**

 Update θ by running the training algorithm for n steps

$i \leftarrow i + n$

$v' \leftarrow \text{ValidationSetError}(\theta)$

if $v' < v$ **then**

$j \leftarrow 0, \theta^* \leftarrow \theta, i^* \leftarrow i, v \leftarrow v'$

end

else

$j \leftarrow j + 1$

end

end

return θ^*, i^*

θ_o — initial parameters

p — patience (how many bad evaluations to tolerate before stopping)

n — steps between evaluations (how often to check validation performance)

Outputs:

θ^* — the best parameters found

i^* — the number of training steps at which they were found

Initialise:

$\theta \leftarrow \theta_0$ (current parameters = starting point)

$i \leftarrow 0$ (step counter)

$j \leftarrow 0$ (counter for consecutive non-improvements)

$v \leftarrow \infty$ (best validation error so far — starts at infinity)

$\theta^* \leftarrow \theta, i^* \leftarrow i$ (best parameters found so far)

Main loop runs while $j < p$ (patience not exhausted)

Update θ by running gradient descent for n steps, then update step counter:

$i \leftarrow i + n$

Evaluate on validation set:

$v' \leftarrow \text{ValidationSetError}(\theta)$

Check for improvement:

If $v' < v$ (validation error improved):

Reset patience counter: $j \leftarrow 0$

Save this as the new best: $\theta^* \leftarrow \theta, i^* \leftarrow i, v \leftarrow v'$

Else (validation error did not improve):

Increment patience counter: $j \leftarrow j + 1$

The loop continues until j reaches p , p evaluation periods without any improvement.

Why track θ separately from θ^* ?

By the time you decide to stop (patience exhausted), the current θ might already be overfitting.

So the algorithm keeps a memory of the best θ seen along the way and returns that instead.

Binary "Linear" Classification: Generalized Linear Model

GLM is a flexible statistical technique used to predict outcomes based on multiple variables.

Suppose $Y \in \{0, 1\}$ — your outcome is binary (e.g. "is this a face?")

Since Y is binary (classification), $E[Y|X] = P[Y=1|X]$, so predicting the expected value of Y is the same as predicting the probability it's 1

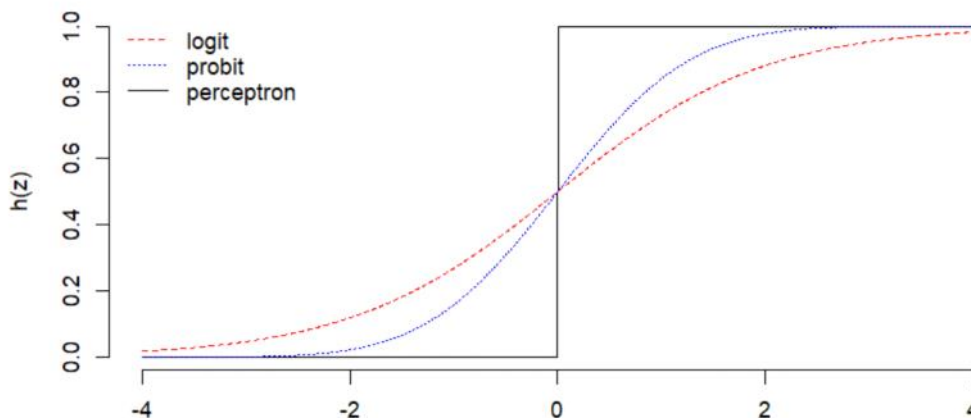
This probability is modelled as $h(X'\beta)$ — a linear combination of inputs $X'\beta$, squashed through a function h .

$X'\beta$ can be any real number ($-\infty$ to ∞), but a probability must be in $[0,1]$. h is called the inverse link function — its job is to map the raw linear score into a valid probability.

Three choices of h given on the slide:

- logit: $h(z) = \frac{1}{1+\exp(-z)}$
- probit: $h(z) = \Phi(z)$
- perceptron: $h(z) = \mathbf{1}_{z>0}$

What they would each look like:



CDF of the standard normal, also S-shaped, similar to logit (sigmoid) but slightly different tails

Maximum likelihood estimation

How you estimate β from the model above.

To fit a model, you need to specify a distribution for y given x . Y is binary so you only need $P[Y_i=1|X_i]$ and this single number fully describes a Bernoulli distribution.

A trick with Bernoulli:

- $\mathbb{P}[Y_i|X_i] = (h(X_i'\beta))^{Y_i} (1 - h(X_i'\beta))^{1-Y_i}$

If $Y_i = 1$, this collapses to $h(X_i'\beta)$. If $Y_i=0$, it collapses to $1-h(X_i'\beta)$. One formula, both cases.

For i.i.d. data, the joint probability of seeing all your data is the product of individual probabilities: $\prod_i P[Y_i|X_i]$

As often is the case, products are awkward to differentiate/optimize and numerically unstable for large n so you take logs, turning it into a sum.

$$\begin{aligned}\frac{1}{n} \log \left(\prod_{i=1}^n \mathbb{P}[Y_i|X_i] \right) &= \frac{1}{n} \sum_{i=1}^n \log (\mathbb{P}[Y_i|X_i]) \\ &= \frac{1}{n} \sum_{i=1}^n Y_i \log (h(X_i' \beta)) + (1 - Y_i) \log (1 - h(X_i' \beta))\end{aligned}$$

choose β to maximize $\mathcal{L}$ — this is the parameter estimation procedure. It's like parameter estimation to least squares for linear regression but for binary outcomes.

Gradient of Logit MLE

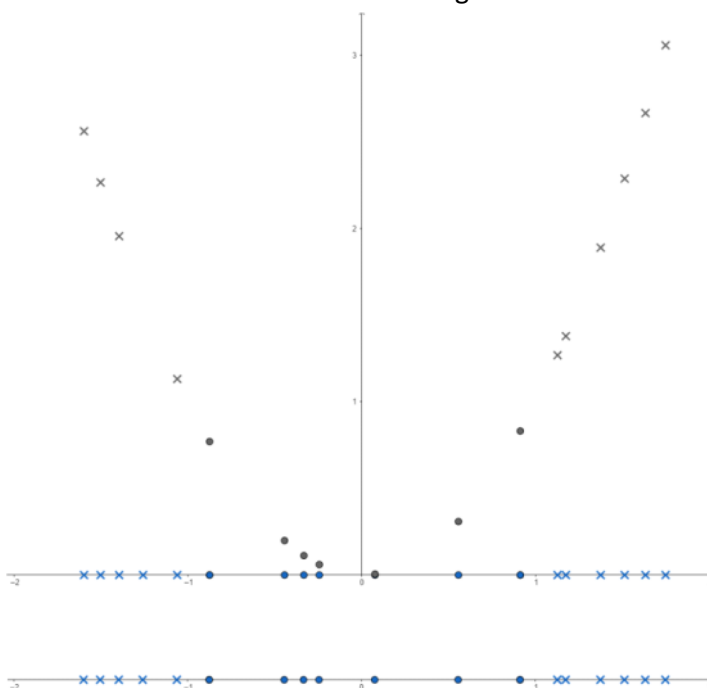
For logit: $h(z) = 1/(1 + \exp(-z))$, $h'(z) = h(z)(1 - h(z))$. Hence

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \beta_j} &= \frac{\partial}{\partial \beta_j} \frac{1}{n} \sum_{i=1}^n Y_i \log (h(X_i' \beta)) + (1 - Y_i) \log (1 - h(X_i' \beta)) \\ &= \frac{1}{n} \sum_{i=1}^n Y_i \frac{\partial}{\partial \beta_j} \log (h(X_i' \beta)) + (1 - Y_i) \frac{\partial}{\partial \beta_j} \log (1 - h(X_i' \beta)) \\ &= \frac{1}{n} \sum_{i=1}^n Y_i \frac{h'(X_i' \beta) X_{i,j}}{h(X_i' \beta)} + (1 - Y_i) \frac{-h'(X_i' \beta) X_{i,j}}{1 - h(X_i' \beta)} \\ &= \frac{1}{n} \sum_{i=1}^n h'(X_i' \beta) X_{i,j} \left(\frac{Y_i}{h(X_i' \beta)} - \frac{1 - Y_i}{1 - h(X_i' \beta)} \right) \\ &= \frac{1}{n} \sum_{i=1}^n (Y_i - h(X_i' \beta)) X_{i,j} \rightarrow \nabla \mathcal{L} = \frac{1}{n} X' (Y - h(X\beta)) = \frac{1}{n} X' (Y - \hat{Y})\end{aligned}$$

Just note that only for logit function $h(z) = 1/(1+\exp(-z))$, its derivative has a beautifully simple closed form: $h'(z) = h(z)(1 - h(z))$

The punchline is that the gradient is just X' times the residual, scaled by $1/n$, same form as OLS gradients. Despite h being non-linear, derivatives cancel out.

Another idea introduced is that adding non-linear features can rescue a linear classifier.



For instance, on the bottom line there doesn't seem too be a way to classify the points.

But when you add another dimension, into 2D, now it looks like x^2 and the classes can be separated; a problem that is not linearly separable in the original features can become separable with the right nonlinear features.

Instead of just adding x^2 , generalise, so add squares + interactions, cubes + interactions, all the way to order p . The Weierstraß approximation theorem is cited as the theoretical justification: any continuous function can be approximated arbitrarily well by a polynomial, given a high enough order. So in principle, polynomial expansion can solve any problem.

The catch is parameter scaling. For a 400 x 600 image, you have 240k features in a plain linear model.

If you expand to order 2 (pairwise interacts between pixels) the number of parameters explodes to $240k(240k-1)/2 = 28.8$ billion

That's computationally infeasible to even store, let alone fit. This is the practical death of the graph above's idea.

The proposed fix is kernel methods. You use Ridge Regression's solution: $\hat{\beta}^R = (X'X + \lambda I_p)^{-1}X'y$

$$\begin{aligned}\hat{\beta}^R &= (X'X + \lambda I_p)^{-1}X'y \\ &= \phi(X) (\phi(X)' \phi(X) + \lambda_n)^{-1} y = \phi(X) (K + \lambda_n)^{-1} y\end{aligned}$$

so that

$$\begin{aligned}\hat{y}^R(x) &= \phi(x)' \phi(X) (K + \lambda_n)^{-1} y \\ &= \kappa(x) (K + \lambda_n)^{-1} y\end{aligned}$$

$K = \phi(X)' \phi(X)$ is the kernel (Gram) matrix.

$\kappa(x)$ is the kernel between x and all training points.

Top line can be rewritten as the second line.

Now we never need to compute $\phi(X)$ explicitly; you only need the inner products, which a kernel function can compute directly and cheaply.

Predictions for a new point (last two lines) follow that logic.

The issue here though is our own scaling problem that the kernel matrix is $n \times n$ so it scales badly with the number of data points, not features like before though.

The features are also hand-picked, not learnt. As in we just manually introduced x^2 , this was not learnt.

We will see that NNs learn its own nonlinear feature representations via hidden layers.

We find out that the GLM from before is actually a trivial NN, will see what happens when we add a hidden layer.

$\hat{y}^{\text{glm}} = h(X\beta)$ — the exact model from before.

It has zero hidden layers meaning input X goes straight into the activation function h to produce an output, the simplest case. Generalising to one hidden layer:

$$h^1 = g^1(XW^1 + b^1)$$

$$\hat{y} = g^2(W^2h^1 + b^2)$$

where

- W^1 are the "weights" in the first hidden layer
- b^1 are the "biases" in the first hidden layer
- g^1 is the first "activation function"
- W^2 are the "weights" in the second hidden layer
- b^2 are the "biases" in the second hidden layer
- g^2 is the second "activation function"

Now we take raw input x , apply a linear transform then squash it through an activation function g^1 producing a new learned set of features h^1 , the hidden layer.

Then take those hidden features h^1 and run the exact same kind of operations again to get the final output $\hat{y}$.

So a NN is a GLM whose output feeds into another GLM.

You compute h^1 first, then feed that into $\hat{y}$.

In a standard $y = mx + b$, m is like the weight and b the bias. M tells you how much y changes when x changes (importance/sensitivity of that input). b is a constant added regardless of x .

At the start, W^1 , b^1 , W^2 , b^2 are just random numbers — the network knows nothing. "Learning" means adjusting all of these numbers so that $\hat{y}$ gets closer to the true y .

Note: A neuron is the basic computational unit that mimics a biological brain cell. It receives an input, processes it by multiplying the inputs by learned weights, adds a bias and passes the result through an activation function g .

One neuron = one weight per input + one bias, total.

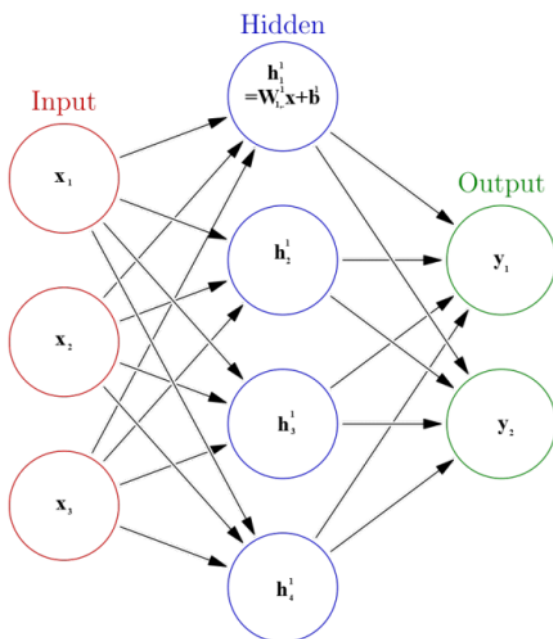
A hidden layer is several neurons all looking at the same inputs each forming its own opinion with its own weights and its own bias.

How the weights and biases come about is later in the deck.

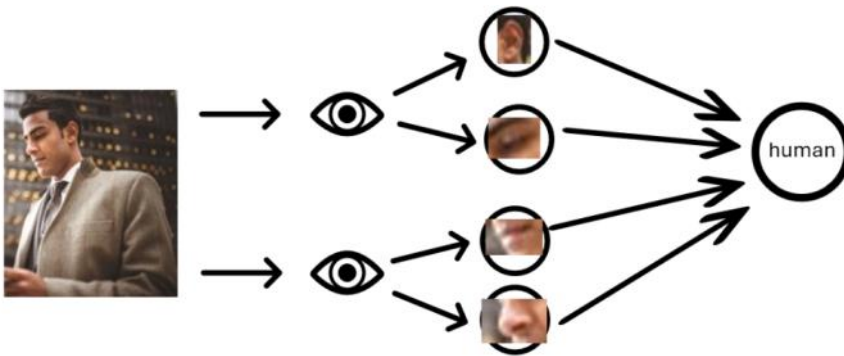
So now a neuron receives inputs eg x_1, x_2, x_3 . Each one gets its own weight, telling you how much that particular input matters.

The whole idea is that we wanted nonlinear features to make classification work, but manually expanding into polynomials exploded combinatorially and the kernel-trick had its own scaling issues.

The hidden layer h^1 ; instead of us deciding the nonlinear features, the network learns W^1 and b^1 from the data, discovering whatever nonlinear transformations of the input are useful.



Inputs into hidden layers, outputs. Each input going into each hidden neuron and every hidden neuron to every output. h^1 shows a weighted sum + bias. Each hidden layer has its own weights and biases.



Recognition can happen hierarchically.

Simple, local patterns get detected first and then combined into a higher-level judgement.

Notation:

$$h^1 = g^1(W^1x + b^1)$$

$$h^2 = g^2(W^2h^1 + b^2)$$

⋮

$$h^L = g^L(W^Lh^{L-1} + b^L)$$

$$\hat{y} = g^o(W^oh^L + b^o)$$

This generalises the 'one hidden layer' example to many.

Same pattern repeated L times, the number of layers, this is called the depth.

Number of neurons in a given layer ($\text{nrow}(W^1)$) is called the width. Meaning number of rows in the matrix are how many neurons there are. Since each row of W^1 is the weight vector belonging to one specific neuron, to produce one value per neuron you need one row per neuron.

So $\text{nrow}(W^l)$ = number of rows in the weight matrix = number of neurons in layer l.

Last layer is the output layer, x is called the input layer.

NN (common) activation functions, the choices for g

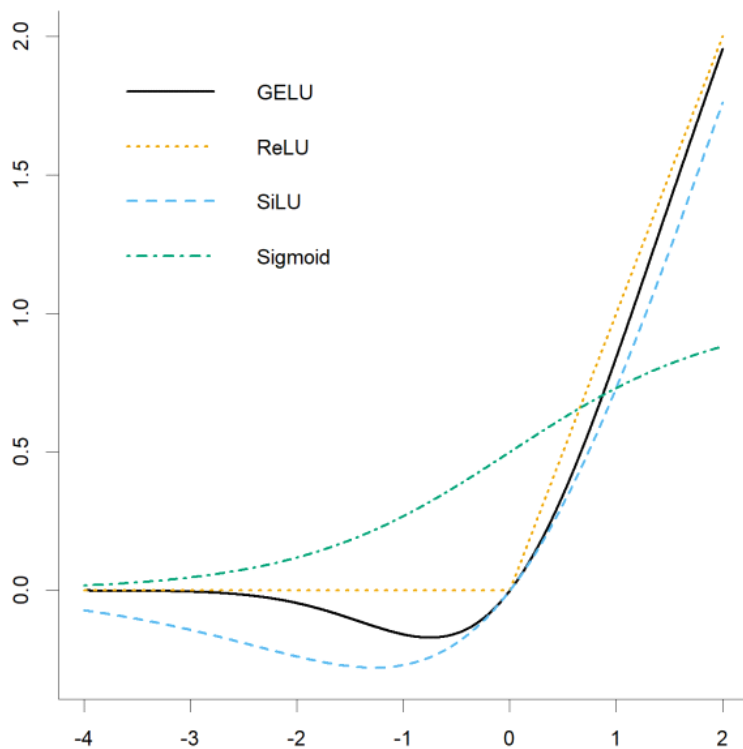
$$f^{\text{GELU}}(x) = x\Phi(x)$$

$$f^{\text{ReLU}}(x) = \max\{0, x\},$$

$$f^{\text{sigmoid}}(x) = 1/(1 + \exp(-x)),$$

$$f^{\text{SiLU}}(x) = xf^{\text{sigmoid}}(x),$$

What they look like:



Horizontal axis (x) axis is the input to the activation function (pre-activation value).

Vertical axis (y) is the output of the activation layer.

Sigmoid is the one curve that flattens out and stays bounded between 0 and 1.

ReLU is 0 for any negative output then rises linearly, slope 1.

GELU and SiLU are smoother versions of ReLU, curving towards 0 instead of sharply.

Squashing, used before, just means compressing a large number that could be very positive or negative into a smaller, controlled range, eg sigmoid squashes it into (0,1).

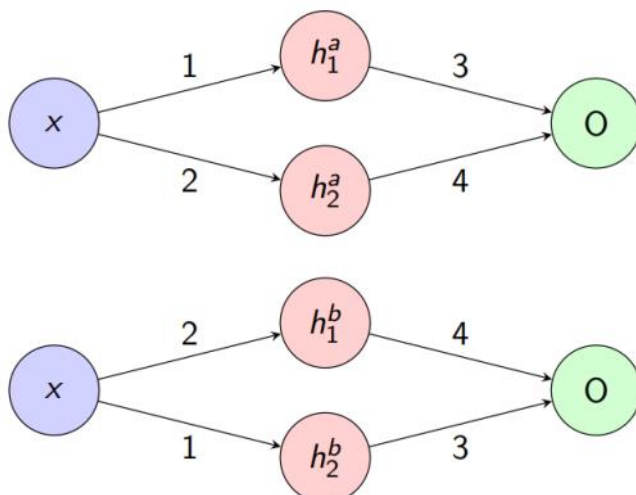
Universal Approximation Theorem

Any continuous function can be approximated arbitrarily well by a neural network, as long as you allow it to be arbitrarily wide or arbitrarily deep.

Two separate routes to the same guarantee -- you can get there by adding more neurons to a single hidden layer (width) or by stacking more layers (depth). NN version of the before with series expansion.

But:

Says nothing about 'how' wide or deep to actually go, whether gradient descent will actually find those weights, how much data you'd need to learn and whether the result generalises well rather than overfitting.

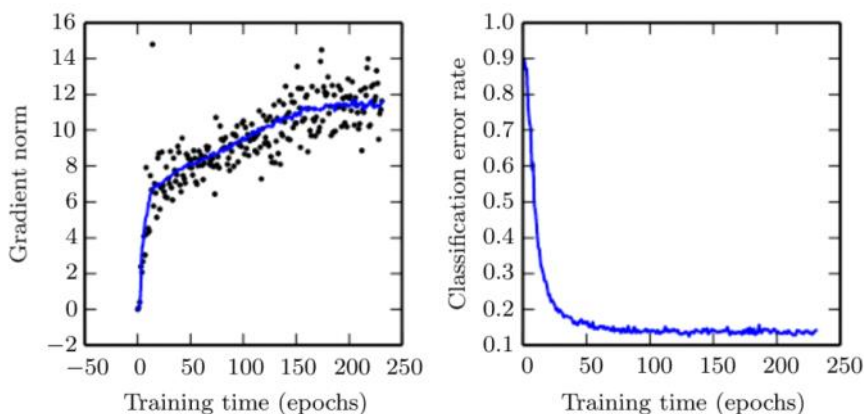
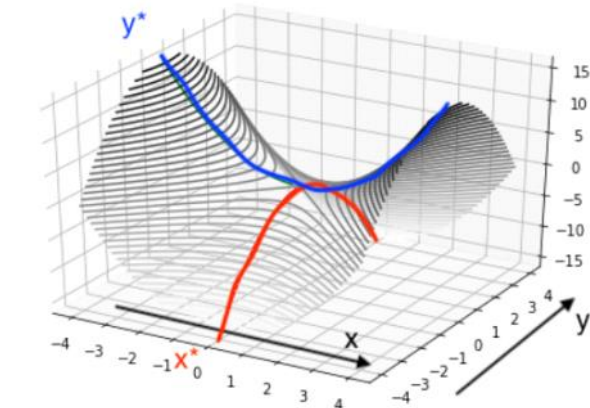


All this says is with two networks, a and b, if you swap the two hidden neurons, you still get the same answer. You can swap the identities of two different neurons within a layer (and their corresponding weights/biases) and produce a network that's functionally indistinguishable. If a hidden layer has n neurons, there are actually $n!$ (n factorial) different ways to permute/relabel those neurons that all give you the exact same overall function.

NNs are not convex. There may be local minima, saddle points (are like inflection points, where the derivatives in orthogonal directions are zero but it is neither a local max nor min. It's like a pringles chip middle point. Finding a global min is very hard.

Stacking layers and nonlinear activations creates a highly non-convex loss surface.

Saddle point



Epochs are complete passes through the training set. You go through the training set multiple times because each individual gradient update only nudges the weights a tiny amount to stay controlled and not destabilise, which start as essentially random values. Given that the loss surface is non-convex and bumpy, getting from a random starting point to good low-loss region requires many small steps and not just one batch's worth.

Classification error rate behaves as expected, dropping sharply early on and then levelling off around 0.10-0.15. You'd normally expect the gradient to shrink toward zero as you approach a minimum, instead the gradient norm keeps growing while the actual error keeps improving. This is the signature of ill-conditioning, the loss surface has wildly different curvature in different directions so gradient descent move inefficiently, often oscillating across steep directions while crawling along flat ones.

The last few slides show that you can't even uniquely identify 'the' solution due to symmetry, aren't guaranteed to find a global optimum, and even basic gradient descent struggles locally due to ill-conditioning.

The motivation for later sophisticated optimisers.

What is currently popular (as of the slides) SiLU/Swish, Mish and similar smooth ReLU-like variants. Transformers specifically like BERT, GPT use GELU.

Output layer choices:

Linear layer: no squashing at all, the raw weighted sum passes straight through. Used for regression where your target y can be any real number and wouldn't want to artificially bound the output like sigmoid or ReLU would.

Softmax layer:

$$\bullet \sigma(z)_j = \frac{\exp(z_j)}{\sum_{k=1}^K \exp(z_k)}$$

This takes K raw scorers and converts them into a valid probability distribution, every output is positive and they all sum to 1.

Cost functions, loss function choices:

For regression, MSE:

$$\frac{1}{n} \sum_{i=1}^n (y_{i,k} - \hat{y}_{i,k})^2$$

For classification: cross-entropy

$$-\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k})$$

The cross-entropy method is just the negative of the log-likelihood from before, generalised to K classes.

Training a classification network by minimising cross-entropy is mathematically identical to the MLE procedure (max likelihood is the same as minimising negative log-likelihood).

Regression uses a linear output layer + MSE; classification uses softmax + cross-entropy

Regularisation:

Can do an l_p penalty (usually L2)

Early stopping

Dropout, where during each training update, randomly set some neurons' outputs to zero (tuneable rate) to prevent the network from becoming overly dependent on any specific neuron or combination. Spread useful information across the network.

Parameter sharing, reusing the same weights in multiple places reduces the number of independent parameters the network has to learn and fewer free parameters means less room to overfit.

(Adversarial) noise injection, add small perturbations to the inputs during training (sometimes specifically worst-case/adversarial ones) forcing the network to be robust to tiny variations rather than memorising exact training examples.

Prediction with a NN

Suppose we are handed everything, structure, weights, biases, activation functions g (all fixed).

Given a new input x , how to turn that into a prediction $\hat{y}$?

$$\begin{aligned}
h^1 &= g^1(W^1x + b^1) \\
h^2 &= g^2(W^2h^1 + b^2) \\
&\vdots \\
h^L &= g^L(W^Lh^{L-1} + b^L) \\
\hat{y} &= g^o(W^o h^L + b^o)
\end{aligned}$$

Compute each layer's output sequentially, left to right, feeding each result into the next layer until getting the final prediction. This is the feedforward pass. Just a chain of matrix multiplications.

The hard part of NNs isn't using them, it's finding the right weights in the first place.

Intermediate values h^l are useful for computing gradients, when eventually we need to train the network we will see this again.

Training

We will be using (stochastic) gradient (in the optimisation slides) descent but in next year's course we will see more specialised methods.

Adam (Kingma and Ba, 2014) is the most popular as of the slides.

Choosing a good model architecture matters more than choosing a fancy optimiser. Most progress in NNs has come from changing the model not the optimiser.

SGD Three workings parts:

Starting value at a random initialisation. If neurons start identical, they stay indistinguishable forever, as seen near the start).

Computation of the derivative, backpropagation. Name of the methods, will see later.

Step size:

Classical means a fixed or manually scheduled learning rate; adaptive means the optimisers which automatically adjust the step size per parameter, directly addressing the ill-conditioning problem from before where a single fixed step size struggles when the loss surface has very different steepness in different directions.

Computing the gradient:

First, why backpropagation is needed.

Backpropagation (short for backward propagation of error) is the algo used to train NNs. It calculates how much each connection weight contributed to the network's final prediction error, allowing the system to update parameters continuously to improve.

A neural network is just a long composition of linear and nonlinear functions stacked together

Differentiating a composition like $f(g(x))$ requires the chain rule but rather than recomputing everything from scratch for each parameter, reuse the intermediate values computed in the forward pass (the note before which said intermediate values h^l are useful for computing gradients)

Backprop:

Do the feedforward pass first, storing every h^1 .

Compute the derivative at the output layer first (w.r.t. $\hat{y}$ and the output parameters)

Then move to the last hidden layer

Then the second-to-last hidden layer

...continue all the way back to the first hidden layer

Libraries change but principles don't.

Industry is leapfrogging libraries, between pytorch, tensorflow, JAX, etc.

Plenty of optimisation under the hood is possible to, eg with the GPU.

Can find pre-trained models: <https://huggingface.co/models>

If someone trained a NN for detecting and classifying objects, eg cats. But you are asked to classify satellite images of villages in developing into flood vs no flood, then the idea is to 'reuse' someone else's pre-training.

Rather than starting at random initialisation, take the already-trained NN as the starting point.

Replace just the output layer (since the original model was built for a different set of classes), swap in a new output layer sized for your classes and initialise that part randomly.

Then run only a "few" additional gradient steps to adapt the parameters to your new dataset, rather than the full training process.

Saves time, computation, electricity -- you are skipping almost all of the expensive part.

It often performs better than training from scratch on a small dataset which would overfit. A pre-trained model has already learned generically useful low-level features.

the term given to these large, general-purpose pretrained networks that serve as a reusable starting point for many different downstream tasks are Foundational Models.

Hugging Face is a company and online platform that's become the main hub for sharing pretrained machine learning models — think of it as something like GitHub, but specifically for ML models (and datasets). Alongside Python libraries, most famously 'transformers' that makes loading and using these models a few lines of code.

People post models because in research sharing your work/trained model alongside a paper is now close to an expectation, to let others verify, build on and cite your work. It is also just a part of the open-source ML culture. Organizations like Meta or Google sometimes release models openly to encourage developers to build on their tools (driving adoption of their broader ecosystem), to set a de facto standard, or simply for goodwill and recruiting/marketing purposes.

Unsupervised Learning

09 June 2026 00:49

Supervised learning (what NN and before was about) is: $(x_i, y_i) \rightarrow P[Y|X], E[Y|X]$.

Eg with NNs you have pairs of inputs and labels and the goal was to model $P[Y|X]$ (the probability of the label given the input).

Unsupervised learning: $(x_i) \rightarrow P[X]$. You only have the inputs — no labels at all. There's no "y" to predict; instead, the goal is to understand the distribution of X itself (eg how the data is structured, spread, with nothing telling you what is correct).

Three applications, each targeting a different feature of $P[X]$:

Clustering: separate the 'support' of $P[X]$. Support means the regions where the data actually has non-zero probability. If the data clusters into naturally formed groups, clustering tries to identify those groups without being told which points belongs to which groups.

Anomaly detection: find x's with unusually low $P[X = x]$. If you've estimated how "typical" different values of x are, an anomaly is simply a point that's very improbable under that distribution

Dimensionality reduction — find a lower-dimensional z that still captures something meaningful about the original x.

Link to supervised learning:

- $x \in \mathbb{R}^d, x = (x_1, \dots, x_d)$
- d may be large, e.g. images, video, text
- then

$$\begin{aligned} & \mathbb{P}[X = x] \\ &= \mathbb{P}[X_1 = x_1, X_2 = x_2, \dots, X_d = x_d] \\ &= \mathbb{P}[X_1 = x_1 | X_2 = x_2, \dots, X_d = x_d] \mathbb{P}[X_2 = x_2, \dots, X_d = x_d] \\ &= \mathbb{P}[X_1 = x_1 | X_2 = x_2, \dots, X_d = x_d] \mathbb{P}[X_2 = x_2 | X_3 = x_3, \dots, X_d = x_d] \\ & \quad \mathbb{P}[X_3 = x_3, \dots, X_d = x_d] \\ &= \prod_{j=1}^d \mathbb{P}[X_j = x_j | X_{j+1} = x_{j+1}, \dots, X_d = x_d] \end{aligned}$$

- link to generative AI: predict next word

The seemingly hard unsupervised task of modelling the full joint distribution of $P[X]$ can actually be broken down into a sequence of ordinary supervised learning problems.

x is d-dimensional vector.

For images, video or text d can be huge, eg every pixel, frame/word/token.

Repeatedly applying the definition of conditional probability: $P[A \cap B] = P[A|B] \cdot P[B]$:

Start with the full joint probability:

$$P[X=x] = P[X_1=x_1, X_2=x_2, \dots, X_d=x_d]$$

Peel off first variable, conditioning on the rest. Then the second, from what's left. So on all the way down to a single variable.

The end result is the final line of working.

So the entire joint probability is just a product of conditional probabilities, one per coordinate, each conditioned on the remaining coordinates.

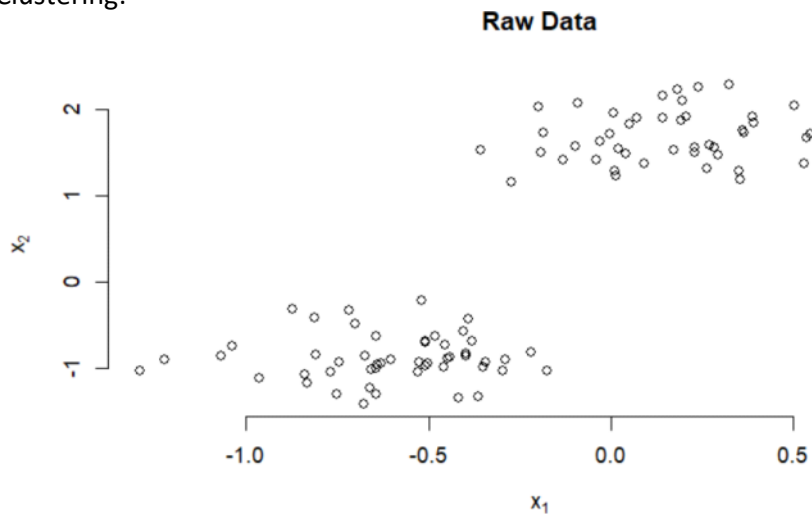
Now each part, $P[X_j=x_j | X_{j+1}, \dots, X_d]$ looks like the first thing we wrote which is supervised learning.

So density estimation (unsupervised) decomposes into a chain of prediction problems (supervised), each solvable with the same tools — logistic regression, neural networks etc.

This is how LLMs work, by predicting the next word.

Treat a sentence as a sequence of tokens, modelling the probability of the entire sentence reduces, via this chain, to repeatedly solving 'predict the next token, given everything else so far'.

Clustering:



Data points form two distinct parts even without labels. We will see the K-means algorithm next.

k-means clustering, formalising the clustering illustration above.

You're given n data points (x_i) , and you want to partition them into k groups $S_1, \dots, S_k$ to minimize the within-cluster sum of squares:

$$\arg \min_{S_1, S_2, \dots, S_k} \sum_{j=1}^k \sum_{x \in S_j} \|x - m^j\|_2^2 = \arg \min_{S_1, S_2, \dots, S_k} \sum_{j=1}^k |S_j| \text{Var}(S_j)$$

where m^j is the centroid (mean) of cluster j

$$m^j = \frac{1}{|S_j|} \sum_{x \in S_j} x,$$

$|S_j|$ is the size of S_j .

So minimising the squared sum distance of every point from its cluster, then sum that up across all k clusters.

This can be written as: $\sum_j |S_j| \cdot \text{Var}(S_j)$

Cluster size multiplied by the variance of points within that cluster. Just minimising the (size-weighted) variance.

Lloyd's algorithm (how to solve it):

k-means is NP-hard, trying every possible way to partition n points into k groups is computationally intractable as n grows. Realistically use a heuristic that finds a good (enough) solution.

Initialise: pick some starting centroids for each of the k clusters:

$$m_{j=1, \dots, k}^{(0,j)}$$

randomly placed. Like the random weight initialisation with NNs.

Repeat until convergence, alternating between the assignment step and the update step.

Assignment step: for each point x^i , assign it to whichever centroid is currently nearest (smallest squared distance):

$$L_i^{(t)} = \arg \min_{j=1, \dots, k} \|x^i - m^{(t,j)}\|^2$$

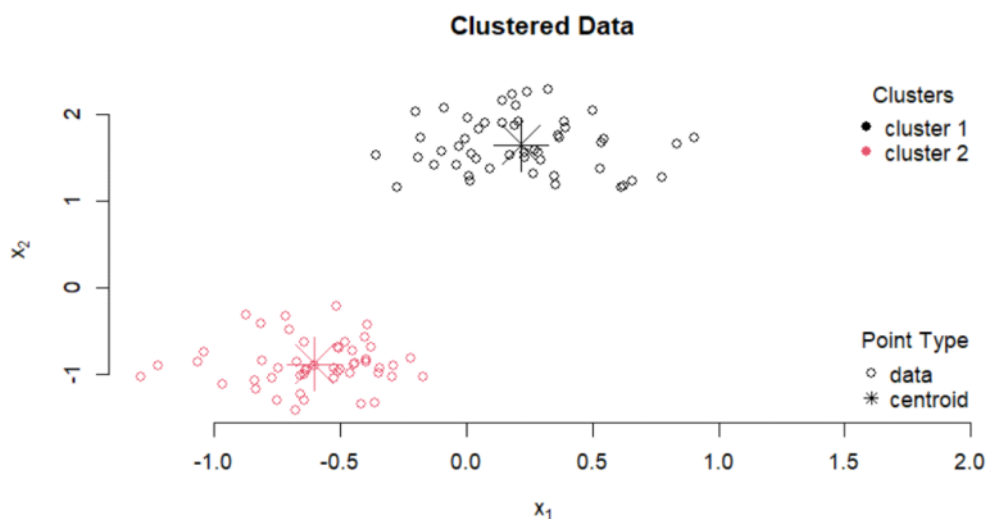
Update step: recompute each centroid as the mean of all points now assigned to it: $m^{t,j} = \text{average of } x_i \text{ over all } i \text{ in cluster } j$

Each round, you first relabel points based on the current centroids, then move the centroids to the actual centre of their newly assigned points**. Centroids shift → some points become closer to a different centroid → reassign again → centroids shift again → and so on, until assignments stop changing (convergence).

Never makes the objective worse at each step but due to NP-hardness it can still get stuck in locally good, but not global optimal, clustering, depending on where the initial centroids happened to start.

** means upon the first spread out centroids, all points are labelled with what their current closest centroid is.

Then eg on the first round, move the centroid into the middle of its cohort of points, given that it has moved, some points will no longer be assigned that centroid as closest, but some will assign to it. Then realign again.



Now k-means has, without labels, grouped the data.

Dimension reduction

We wrote before:

Dimensionality: come up with z in lower dimension than x that are similar to x in a specific way. Similar means preserving distances between points while lowering the dimension.

Lemma

Given $0 < \varepsilon < 1$, a matrix $X \in \mathbb{R}^{n \times p_{\text{origin}}}$, $p_{\text{target}} > \frac{8 \ln(n)}{\varepsilon^2}$, there is a matrix $F \in \mathbb{R}^{p_{\text{origin}} \times p_{\text{target}}}$ such that

$$(1 - \varepsilon) \|X_{i_1, \cdot} - X_{i_2, \cdot}\|_2^2 \leq \|X_{i_1, \cdot} F - X_{i_2, \cdot} F\|_2^2 \leq (1 + \varepsilon) \|X_{i_1, \cdot} - X_{i_2, \cdot}\|_2^2$$

for all $i_1, i_2 \in \{1, \dots, n\}$.

You want to compress each point down to a smaller dimension, p_{target} .

The lemma: as long as you choose p_{target} large enough, specifically, $p_{\text{target}} > 8 \ln(n)/\epsilon^2$, there's guaranteed to exist a projection matrix F such that for every pair of points i_1, i_2 , in your dataset, their squared distance after projection is within a factor of $(1 \pm \epsilon)$ of their squared distance before projection (the long centred line above).

So all pairwise distances are nearly preserved, just at a much lower dimension.

The required target dimension p_{target} depends only on the number of points n and your desired accuracy ϵ — it doesn't depend on p_{origin} (original dimensionality) at all, and only grows logarithmically with n .

You don't need to specifically make F .

Fill F with i.i.d. standard normal random entries, then divide by $\sqrt{p_{\text{target}}}$ to normalize the scale. A random matrix, with the right scaling, works with high probability

This is very different philosophy from PCA/SVD next slide, which carefully computes an optimal data-dependent projection.

The above is the intuition for word (vector) embeddings in transformers in LLMs.

Even though there's a vast space of possible meanings, words get compressed into relatively low-dimensional embedding vectors (hundreds or low thousands of dimensions, not millions), while still preserving meaningful similarity relationships between them.

Principal Component Analysis & SVD

Singular Value Decomposition

Any matrix $M \in \mathbb{R}^{m \times l}$ can be written uniquely as

$$M = U \Sigma V^t$$

where

- $U \in \mathbb{R}^{m \times m}$ is orthogonal, columns are left singular vectors of M ,
- $V \in \mathbb{R}^{l \times l}$ is orthogonal, columns are right singular vectors of M ,
- $\Sigma \in \mathbb{R}^{m \times l}$ rectangular diagonal s.t. $\Sigma_{1,1} \geq \dots \geq \Sigma_{z,z}, z := m \wedge l$.

- $M = \sum_{j=1}^r \Sigma_{j,j} U_{:,j} V_{:,j}^t$ where r is the rank of M

- principal component analysis: stop summing at $\underline{r} \leq r$
- relation to eigendecomposition of $M^t M$ (similar for MM^t)

$$M^t M = (U \Sigma V^t)^t U \Sigma V^t = V \Sigma^t U^t U \Sigma V^t = V \Sigma^t \Sigma V^t$$

Singular Value Decomposition (SVD) as the mathematical foundation of Principal Component Analysis (PCA).
A second, different approach to dimension reduction.

$M = U \Sigma V^t$ says: any matrix M can be broken apart into three simpler pieces multiplied together. Geometrically, applying M to a vector is equivalent to three steps:

First rotate (V^t), then stretch along the axes by different amounts (Σ — the singular values control how much stretching in each direction), then rotate again (U). So instead of M being one complicated transformation, SVD says it's always just "rotate $\rightarrow$ stretch $\rightarrow$ rotate," and U, Σ, V tell you exactly how.

Σ ($m \times l$) is "rectangular diagonal" — nonzero only on the diagonal. These diagonal values are the singular values.

Rewriting M as a sum of building blocks:

$$M = \sum_{j=1}^r \Sigma_{j,j} U_{:,j} V_{:,j}^t$$

This says M can be exactly rebuilt by summing r simple rank-1 pieces, each built from one singular value and a pair of singular vectors. Crucially, because singular values are sorted from largest to smallest, the first terms in this sum capture the bulk of M 's structure, and later terms add progressively less.

This is exactly where PCA comes from: "stop summing at $r \leq r$ " — instead of summing all r terms to reconstruct M exactly, you truncate after just the first few (largest singular value) terms. Approximation of r given far fewer components -- keeping only the directions of greatest variation. Driven by actual structure of data, not random like before.

There is a connection to eigendecomposition.

Since U is orthogonal, $U^t U = I$, so this collapses nicely. This shows V (the right singular vectors of M) are also the eigenvectors of $M^t M$, with eigenvalues equal to the squared singular values ($\Sigma^t \Sigma$). The usual teaching of the derivation of PCA.

Overall,

Johnson-Lindenstrauss gives you a cheap, data-agnostic way to compress dimensions while preserving distances (just draw a random matrix, no need to even look at the data). PCA/SVD instead carefully analyzes your actual data's covariance structure to find the most informative directions — more computationally expensive (you have to compute the decomposition), but typically gives a far more efficient and interpretable lower-dimensional representation for a given target size.

Bandits

09 June 2026 00:50

Reinforcement learning:

Supervised learning has a fixed dataset of labelled examples (x,y) , unsupervised learning has unlabelled data and no feedback signal at all.

RL learns through ongoing interaction, getting delayed, noisy reward signals rather than being told the 'correct' action directly.

RL is a hard problem.

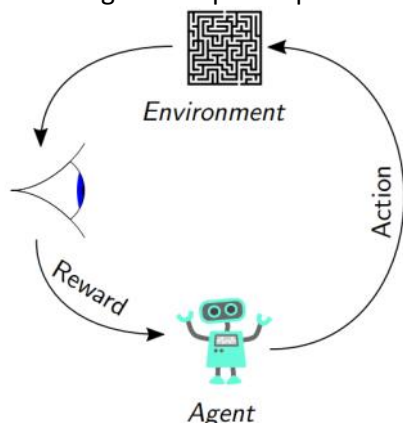
- The number of distinct situations an agent might face is massive. Too many to handle through (brute force) reenumeration.

- Unknown transition probabilities $p(s'|s,a)$. The agent doesn't know in advance how the environment will respond to an action in a given state, must be learned from experience.

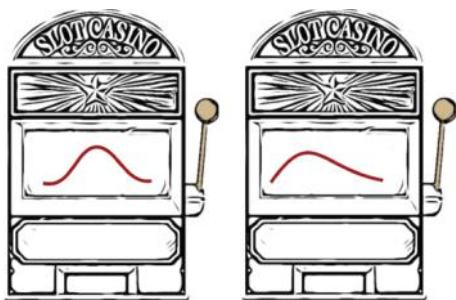
- Need to explore all states/possibilities to learn what's good through attempts. This creates the exploration - exploitation trade-off where do you keep trying new things to learn more or stick with what currently looks best?

- Credit attribution. When a reward arrives, possibly after a long sequence of actions, it's hard to know which of those earlier actions deserved the credit.

Introducing the simplified problem the deck focuses on:



Just actions and rewards repeated. There are no states, which removes the large state space problem.



The agent chooses an action, called an arm, after the slot-machine arm. A multi-armed bandit is a row of several machines, each with a different unknown payout.

Each arm has its own reward distribution, only learn it by trying.

The agent's goal is to maximise the sum of expected rewards across many repeated choices. So you want to figure out, as quickly as possible, which arm has the best average payout and then favour it.

Another example is a worker searching for a job with uncertain total comp.

Formalising

A = number of arms (your set of options)

T = number of periods/trials (how many times you get to choose, the time horizon)

R^a = the random reward from pulling arm a , with $\mu_a = \mathbb{E}[R^a]$ its expected (true average) reward — this μ_a is the underlying quality of arm a , and it's unknown to the agent, which is the whole reason bandit problems are hard.

Optimal arm:

$$a^* = \arg \max_{a=1,\dots,A} \mathbb{E}[R^a]$$

The goal — find a strategy/policy π :

$$\max_{\pi} \sum_{t=0}^T \gamma^t \mathbb{E}[R^{\pi}]$$

A policy π is a rule for deciding what to do next, based on everything observed so far.

The goal is to choose this policy to maximize the discounted sum of expected rewards across all T periods. γ (between 0 and 1) is a discount factor to prioritise near-term payoffs.

Oracle optimal strategy:

If you magically already knew a^* (the true best arm), the obviously best thing to do is play it every single period. Is the theoretical benchmark.

Regret — the key performance measure for the rest of the deck:

$$\sum_{t=0}^T \gamma^t \left(\mathbb{E}[R^{a^*}] - \mathbb{E}[R^{\pi}] \right) \geq 0$$

This sums up, over all periods, the gap between what the oracle would have earned (always $\mathbb{E}[R^{a^*}]$) and what your actual policy π earned, each discounted by γ^t .

How to make the regret as small as possible coming up

Regret decomposition

Theorem

For any policy π , the regret after T periods is

$$R_t = \sum_{a=1,\dots,A} \underbrace{\left(\mathbb{E}[R^{a^*}] - \mathbb{E}[R^a] \right)}_{=: \Delta^a = \text{suboptimality gap}} \mathbb{E}[T^{a,\pi}],$$

where $T^{a,\pi}$ is the expected number of periods that policy π plays arm/action a .

Rewriting the regret formula to sum over arms, not time.

Δ^a , the "suboptimality gap": for each arm a , this is just how much worse that arm's true expected reward is compared to the best arm. If a is the optimal arm, $\Delta^a = 0$. For any other arm, $\Delta^a > 0$, and the size of Δ^a tells you exactly how costly that arm is, per visit.

$T^{a,\pi}$: the expected number of times your chosen policy π ends up playing arm a , out of all T periods.

The intuition: total regret = sum, over every arm, of "how bad is this arm" $\times$ "how often did you play it."

Greedy algorithm

This is the first, simplest and worst performing strategy.

The algorithm: try each arm exactly once (so every arm has at least one data point), then from then on, always play whichever arm currently has the highest observed average reward, updating that average as new data comes in.

There's no exploitation, just exploration.

Instead of storing every single past reward (growing memory use) maintain a running average:

$$\bar{X}_{n+1} = \frac{n}{n+1} \bar{X}_n + \frac{1}{n+1} X_{n+1}$$

Explore-then-commit

The algorithm: try each arm S times (not just once, as in greedy) — a tuneable exploration budget — then, for the rest of the time horizon, permanently commit to whichever arm had the highest average reward during that exploration phase.

Higher S has more thorough exploration but also means more periods deliberately sampling every arm, including bad ones.

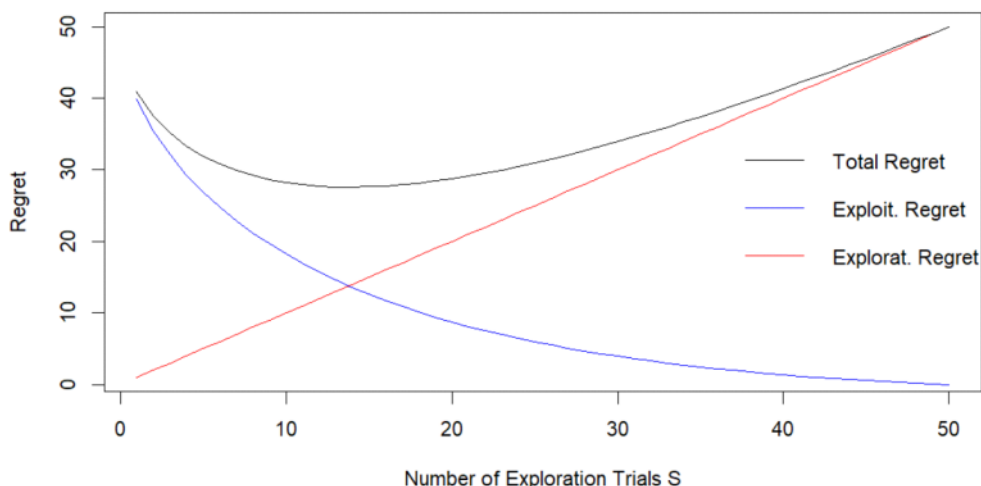
Derivation of probability ETC mistakenly commits to the worse arm, in a case of two arms.

- expected regret: $\Delta_2 S + \Delta_2 (T - 2S) \mathbb{P}[\bar{X}_{2S}^1 \leq \bar{X}_{2S}^2]$

$\Delta_2 S$: the guaranteed cost from exploration itself.

$\Delta_2 (T - 2S) \cdot \mathbb{P}[\text{wrong choice}]$: the expected cost if you commit wrongly, should that happen. You're stuck playing the worse arm for all $T - 2S$ remaining periods, each costing Δ_2 , weighted by how likely that mistake actually was.

Increasing S directly increases the first term (more guaranteed exploration cost) but decreases the second term
The trade-off:



Three curves: red ("Explorat. Regret") rises roughly linearly with S — this is the $\Delta_2 S$ term, the guaranteed cost of exploring the bad arm more. Blue ("Exploit. Regret") falls as S grows — this is the $\Delta_2 (T - 2S) \cdot \mathbb{P}[\text{wrong choice}]$ term, since more exploration lowers the chance of committing to the wrong arm. Black ("Total Regret") is their sum, and it traces a clear U-shape, dipping to a minimum before rising again as S gets too large and you wasted too much time exploring.

Upper confidence bound

Third and most sophisticated we'll see.

Rather than picking the arm with the best current estimate (like before) UCB picks the arm with the best plausible best-case value, given how uncertain you currently are about it, acting optimistically.

The algorithm:

Fix a confidence level, parameter $\delta \in (0,1)$.

Try each arm once (minimal initialization, same as the other algorithms).

For each arm, compute an upper confidence bound B_a — a value such that the true mean only exceeds it with small probability: $\mathbb{P}[E[R^a] \geq B_a] \leq \delta$.

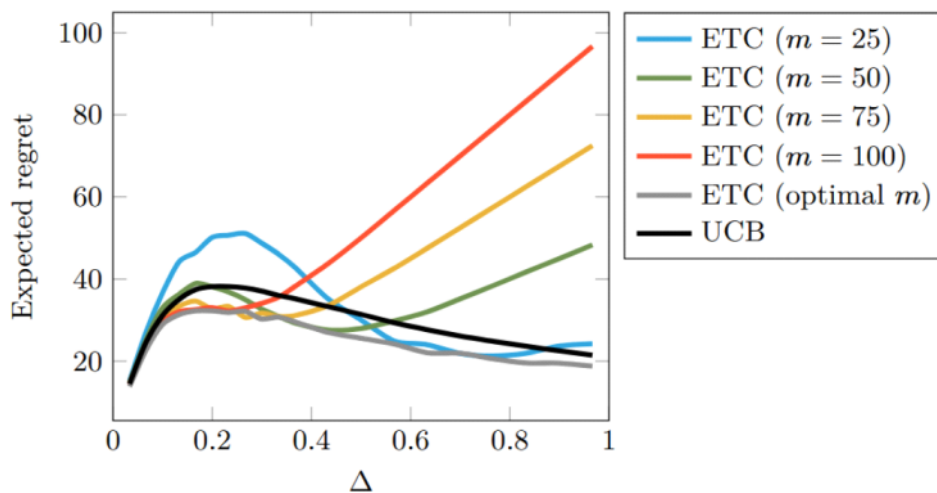
Play the arm with the highest B_a — the optimistic choice.
 Update that arm's bound with the new observation, and repeat.

Why optimism naturally produces exploration: an arm you've barely tried has very little data, so its confidence interval is wide — even if its observed average looks mediocre, its upper bound stays high. This pulls the algorithm toward under-explored arms automatically, without needing a separately tuned exploration phase. Meanwhile, an arm that's been tried a lot and consistently performs poorly will have a tight confidence interval close to its (low) true mean, so its upper bound stays low too, and the algorithm naturally stops choosing it.

Based on $P[E[R^a] \geq B_a] \leq \delta$

A smaller δ requires a stronger guarantee, which means a wider confidence bound — and wider bounds mean more exploration. So mathematically, smaller $\delta \rightarrow$ more exploration, larger $\delta \rightarrow$ less exploration.

Lattimore & Szepesvári's textbook "Bandit Algorithms" — setting $\delta = 1/T^2$ gives regret "close" to optimal (Theorem 7.1), and slightly more refined, time-varying choices of δ can achieve full optimality.



The curves: several ETC curves, each using a different fixed exploration budget m (25, 50, 75, 100 — this is the same parameter called S before).

Axes: x-axis is Δ , the suboptimality gap between the two arms, how different the two arms are (0 is identical, 1 is very different). Y-axis is expected regret accumulated over the full time horizon.

ETC optimal m is if you had oracle knowledge of Δ in advance.

Several ETC curves and then the UCB curve.

for a fixed, small m (e.g. $m=25$), regret is low when Δ is very small (mistakes barely matter) or very large when the gap is obvious and limited exploration understands.

For a fixed, large m (e.g. $m=100$), regret instead climbs steadily as Δ grows, because you're guaranteed to pay $\Delta \times 2m$ in exploration cost regardless of outcome.

The UCB curve tracks almost exactly along the optimal- m envelope (ECB optimal m), across the entire range of Δ , despite never being told Δ or needing to commit to any fixed exploration budget upfront.

Bayesian approaches

Suppose you know the form of each arm's reward distribution, but it depends on an unknown parameter (e.g. you might know rewards are normal, just not the true mean μ_a , like a parametric assumption).

You also have a prior — a belief, before seeing any data, about how likely different parameter values are.

ETC/UCB never assume prior beliefs, just work off observations. The Bayesian approach updates priors and updates it via Bayes' rule as data arrives, producing a posterior distribution over each arm's parameters that blends prior belief with observed evidence.

Gittins Index — the theoretically optimal policy (in a specific setting):

If the time horizon is infinite ($T=\infty$) and rewards are discounted ($\gamma < 1$, so the infinite sum still converges), there's a striking result: the truly Bayes-optimal policy reduces to an index policy — you can compute a single number for each arm separately (based only on that arm's own posterior), and the optimal action is always to play whichever arm currently has the highest index. An explicit formula is available [here](#).

Thompson Sampling — a simpler, widely-used alternative:

Instead of computing confidence bounds explicitly (as UCB does), Thompson sampling randomizes: play arm a with probability equal to the posterior probability that a is actually the best arm.

This naturally balances exploration and exploitation: arms you're still very uncertain about have wide posteriors, so they'll occasionally produce a high random sample purely by chance and get explored; arms you're confident about have narrow posteriors clustered around their true value, so they only get chosen if that value is genuinely good, similar to UCB but through randomisation.

Thompson sampling "enjoys similar optimality properties as optimal UCB under certain conditions" — under the right technical assumptions, it achieves comparably strong (near-optimal) regret, while often being simpler to implement in practice (no need to construct confidence bounds, just sample and compare) — which is part of why it's popular in real applications.

From the deck, UCB comes out on top among the algorithms it directly compares.

The Bayesian alternative, Thompson sampling, is described as achieving "similar optimality properties as optimal UCB under certain conditions" — so roughly comparable performance, just via a different (randomized, posterior-based) mechanism, and often preferred in practice for being simpler to implement. The Gittins Index is theoretically optimal but only in a narrow special case (infinite horizon, discounted rewards), so it's more a notable theoretical result than a general practical recommendation here.

So: UCB or Thompson sampling, in practice — the deck doesn't crown one over the other between those two, just presents them as two strong, theoretically grounded options from different philosophical starting points (frequentist vs Bayesian).